IT3021

Data Warehousing and Business Intelligence

Year 3 · Semester 1 · 2026

Assignment Title	**Assignment 1**
Module	IT3021 -DWBI
Dataset	**Hospital Emergency Room Dataset**
Deadline	27.03.2026
Submission Mode	Report

Content

Task 4: Data Warehouse Design

Task 5: ETL Process Implementation Using SSIS

Task 6: ETL Development for Accumulating Fact Table

TASK 1-Data set Selection and ER-Diagram

1.1 Introduction to the Selected Dataset

For this assignment, the **Hospital Emergency Room (ER) dataset** was selected as the source dataset for designing and implementing the data warehouse solution. This dataset represents real-world hospital emergency room operations and contains raw transactional data related to patients, visits, and departments.

According to the assignment requirements, the dataset must be a **real-world OLTP dataset**, must not be the lab dataset or AdventureWorks, and must contain sufficient attributes and relationships to support ETL, dimensional modeling, and reporting.

The selected dataset satisfies these requirements because it contains:

- transactional visit-level data,
- multiple related entities,
- measurable attributes,
- and time-based information suitable for analysis.

Therefore, it is appropriate for developing a complete DWBI solution.

Data set size - 9,216 rows × 12 columns

https://www.kaggle.com/datasets/laxdippatel/hospital-emergency-room-dataset

1.2Justification for Dataset Selection

The Hospital ER dataset was selected based on the following reasons:

- It is an **OLTP dataset**, where each record represents an operational event (ER visit), not a pre-built analytical structure.
- It contains **multiple entities** such as patients, visits, and departments, enabling proper conceptual and dimensional modeling.
- It includes **analytical measures** such as Wait_Time and Satisfaction_Score, which are suitable for fact table design.
- It supports **time-based analysis** through admission date.

- It is a **novel dataset**, different from lab examples, fulfilling assignment requirements.
- Additionally, the dataset is suitable for DWBI tasks such as ETL, star schema design, and reporting, as required in the dataset selection guide.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Patient Id	Patient Admission Date	Patient First Inital	Patient Last Name	Patient Gender	Patient Age	Patient Race	Department Referral	Patient Admission Flag	Patient Satisfaction Score	Patient Waittime	Patients CM	
2	145-39-5406	20-03-2024 08:47	H	Glasspool	M	69	White	None	FALSE	10	39	0	
3	316-34-3057	15-06-2024 11:29	X	Methuen	M	4	Native American/Alaska Native	None	TRUE		27	0	
4	897-46-3852	20-06-2024 09:13	P	Schubuser	F	56	African American	General Practice	TRUE	9	55	0	
5	358-31-9711	4/2/2024 22:34	U	Titcombe	F	24	Native American/Alaska Native	General Practice	TRUE	8	31	0	
6	289-26-0537	4/9/2024 17:48	Y	Gionettitti	M	5	African American	Orthopedics	FALSE		10	0	
7	255-51-2877	20-04-2023 00:13	H	Buff	M	58	Asian	None	FALSE		59	0	
8	465-97-0990	23-08-2023 08:26	F	Perrat	F	68	White	None	TRUE		43	0	
9	157-31-7520	29-07-2023 16:57	K	Gwillim	F	47	Two or More Races	None	TRUE		23	0	
10	432-34-5614	19-02-2024 06:54	E	Dewhirst	F	79	White	None	FALSE	1	42	0	
11	609-17-8678	11/10/2024 5:25	M	Crebo	M	62	African American	None	FALSE		51	0	
12	497-14-6812	26-07-2024 01:45	Q	Churchard	F	73	White	Gastroenterology	TRUE		34	0	
13	393-38-9502	10/3/2024 22:02	N	Corpes	F	16	White	Orthopedics	FALSE		39	0	
14	288-05-6370	12/11/2023 16:00	R	Brixey	F	16	Native American/Alaska Native	General Practice	TRUE		53	0	
15	784-54-9931	25-06-2023 09:40	M	Goudie	M	46	Pacific Islander	None	FALSE		45	0	
16	662-21-6522	4/5/2023 13:16	G	Stanlack	M	69	White	None	TRUE		49	1	
17	628-73-1801	19-09-2023 01:53	C	McMurty	M	37	Declined to Identify	None	TRUE		57	0	
18	370-19-2271	25-05-2024 22:11	I	Scothorn	F	50	Asian	General Practice	FALSE		35	0	
19	458-98-8860	25-06-2023 18:59	J	Helgass	M	37	Declined to Identify	None	FALSE		55	0	
20	728-31-2493	4/9/2023 16:15	W	Chittock	F	70	Asian	Physiotherapy	TRUE		50	0	
21	823-34-5523	16-11-2023 23:46	F	Prendergast	M	55	Asian	None	TRUE	2	40	0	
22	621-70-7472	30-06-2023 05:22	T	Bissiker	F	63	Native American/Alaska Native	None	TRUE		25	0	
23	344-36-7156	22-05-2023 16:48	M	Mandell	F	44	Asian	None	FALSE	2	51	1	
24	455-21-3671	17-11-2023 07:24	O	Coste	F	11	Declined to Identify	None	FALSE	4	30	0	
25	259-10-0339	26-01-2024 05:58	Q	Dodridge	M	8	Asian	General Practice	FALSE		16	0	
26	720-54-2625	24-05-2023 14:42	C	Pavie	F	4	Native American/Alaska Native	None	TRUE		23	1	
27	661-92-7059	12/4/2023 21:02	Z	Sleightholm	M	42	African American	None	FALSE	0	51	0	
28	598-53-3927	4/9/2024 2:30	S	Noads	F	68	African American	None	FALSE		58	0	
29	715-74-5338	16-12-2023 13:02	J	Filkin	M	22	Pacific Islander	General Practice	FALSE		25	0	
30	669-74-2146	11/3/2024 17:06	C	Bilby	F	58	Declined to Identify	Orthopedics	FALSE		55	0	

Figure 1.1: Dataset selected one

1.3 Dataset Overview

The Hospital Emergency Room dataset captures information about emergency room operations. The dataset includes:

- Patient-related information
- Emergency room visit records
- Department information
- Operational measures such as waiting time and satisfaction score

The dataset represents a realistic business scenario where:

- patients visit the emergency room,
- visits are handled by departments,
- and each visit produces measurable outcomes.

From this dataset, three main entities were identified:

- PATIENT
- ER_VISIT
- DEPARTMENT

1.4 Data Sources Types

The dataset used in this project is obtained from multiple source types. The primary dataset is provided in CSV format, representing structured transactional data. Additionally, an Excel file is used in later stages (Task 6) to simulate updates to the accumulating fact table.

Therefore, the solution uses:

- CSV files as primary data sources
- Excel files for incremental updates
- SQL Server as the target data warehouse

This ensures the solution meets the requirement of using multiple data source types.

Among these, **ER_VISIT** represents the central transactional event

1.5 Identified Entities and Attributes

Based on the dataset, the following entities and attributes were identified

PATIENT

The PATIENT entity stores descriptive information about each patient.

Attributes:

- Patient_ID *(Primary Key)*
- Name (First_Initial ,Last_Name)
- Gender
- Age
- Race
- Patient_CM

The attribute Name is treated as a composite attribute, consisting of First_Initial and Last_Name.

DEPARTMENT

The DEPARTMENT entity stores information about hospital departments.

Attributes:

- Department_ID *(Primary Key)*
- Department_Name
- Department_Group

ER_VISIT

The ER_VISIT entity represents the emergency room visit transaction and is the central entity in the dataset.

Attributes:

- Visit_ID *(Primary Key)*
- Admission_Date

- Admission_Flag
- Wait_Time
- Satisfaction_Score

In the conceptual ER model, ER_VISIT is connected to PATIENT and DEPARTMENT through relationships.

In the relational implementation, these relationships are represented using foreign keys:

- Patient_ID → references **PATIENT**
- Department_ID → references **DEPARTMENT**

This ensures that each visit is associated with a specific patient and handled by a specific department, maintaining referential integrity and supporting later ETL and data warehouse processes.

1.6 Relationships Between Entities

The relationships between the entities are defined as follows:

PATIENT → ER_VISIT

- One patient can have multiple emergency room visits.
- Each ER visit belongs to one patient.

Relationship**:** PATIENT (1) → ER_VISIT (M)

DEPARTMENT → ER_VISIT

- One department can handle multiple ER visits.
- Each ER visit is handled by one department.

Relationship**:** DEPARTMENT (1) → ER_VISIT (M)

These relationships confirm that ER_VISIT is the central transactional entity in the dataset.

1.7 ER Diagram

Conceptual ER Diagram for the Hospital Emergency Room Dataset

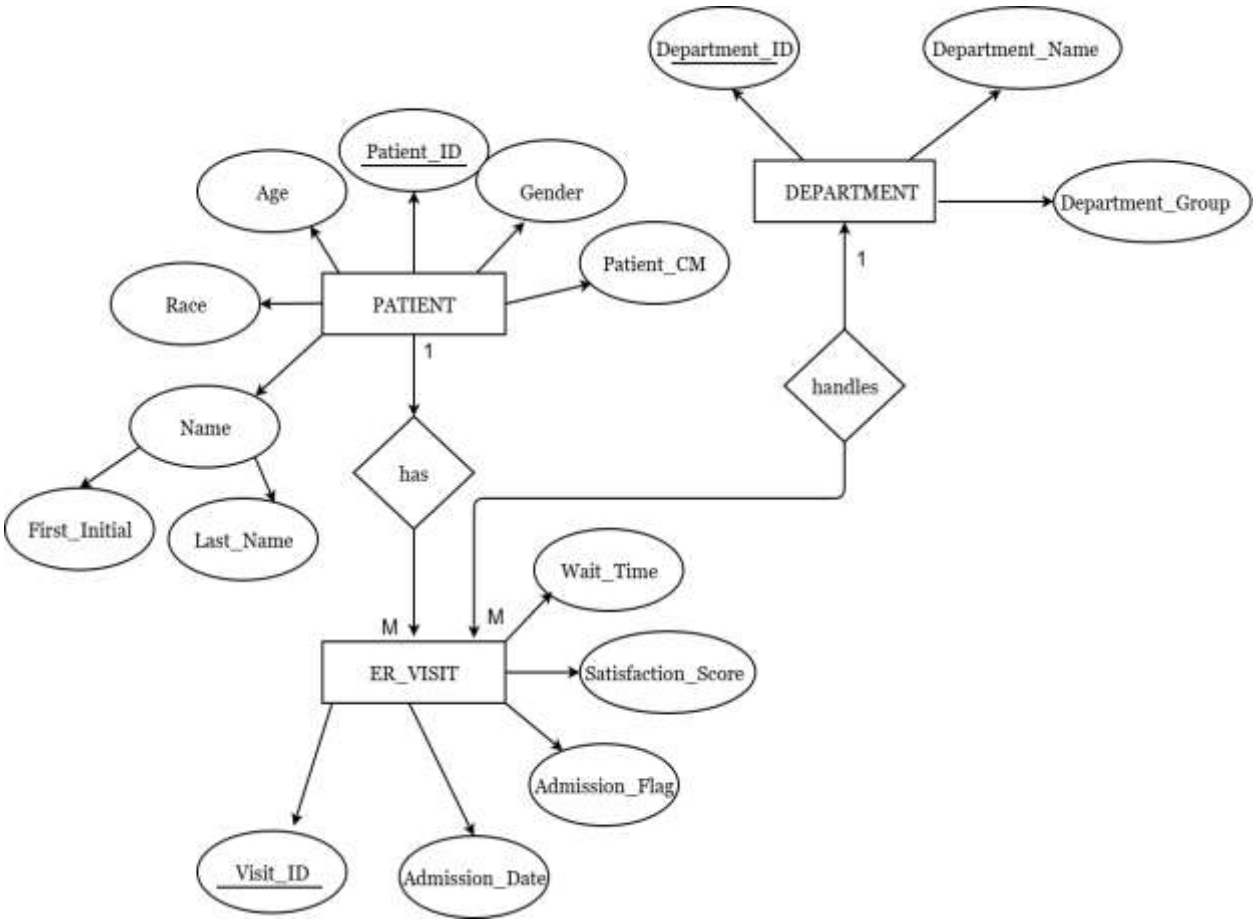


Figure 1.2: Conceptual ER Diagram of Hospital ER Dataset

1.8 Explanation of the ER Diagram

The ER diagram represents the structure of the Hospital Emergency Room dataset using three main entities: PATIENT, ER_VISIT, and DEPARTMENT.

The **PATIENT** entity contains descriptive information such as age, gender, race, and patient classification. The name is modeled as a composite attribute consisting of first initial and last name.

The **DEPARTMENT** entity represents hospital departments responsible for handling emergency visits. It includes attributes such as department name and department group.

The **ER_VISIT** entity represents the main transactional event. Attributes such as admission date, admission flag, waiting time, and satisfaction score describe each individual visit.

The relationships indicate that:

- a patient can have multiple visits,
- and a department can handle multiple visits.

This structure ensures that the dataset is logically organized and suitable for transformation into a dimensional model in later tasks.

1.9 Conclusion

In Task 1, the Hospital Emergency Room dataset was selected and analyzed as the foundation for the DWBI solution. The dataset was confirmed to be an OLTP dataset with sufficient richness and structure.

A conceptual ER diagram was developed to represent the key entities and relationships, identifying ER_VISIT as the central transactional entity linked to PATIENT and DEPARTMENT.

This task provides a strong foundation for the next stages of the assignment, including data source preparation, architecture design, dimensional modeling, and ETL implementation.

TASK 2-Preparation Of Data Sources

2.1 Introduction

Task 2 focuses on preparing the selected dataset for extraction by organizing it into multiple data source formats. According to the assignment requirements, the dataset must be divided into at least two different source types such as CSV, Excel, or database sources.

In real-world data warehousing environments, data is rarely stored in a single system. Instead, data originates from multiple heterogeneous sources. Therefore,

this task simulates a realistic scenario by preparing the Hospital Emergency Room dataset in different formats for ETL processing.

2.2 Overview of Data Sources

For this project, the dataset was divided into multiple source files to represent different data origins. The following data sources were used:

Source 1 – Patient Source (CSV)

- File Name: Patient_Source.csv
- Contains patient-related information Attributes include:
  - Patient_ID
  - First_Initial
  - Last_Name
  - Gender
  - Age
  - Race
  - Patient_CM

	A	B	C	D	E	F	G
1	Patient_Id	PatientFirst_Initial	PatientLast_Name	Patient_Gender	Patient_Age	Patient_Race	Patients_CM
2	145-39-5406	H	Glasspool	M	69	White	0
3	316-34-3057	X	Methuen	M	4	Native American/Alaska Native	0
4	897-46-3852	P	Schubuser	F	56	African American	0
5	358-31-9711	U	Titcombe	F	24	Native American/Alaska Native	0
6	289-26-0537	Y	Gionettitti	M	5	African American	0
7	255-51-2877	H	Buff	M	58	Asian	0
8	465-97-0990	F	Perrat	F	68	White	0
9	157-31-7520	K	Gwillim	F	47	Two or More Races	0
10	432-34-5614	E	Dewhirst	F	79	White	0
11	609-17-8678	M	Crebo	M	62	African American	0
12	497-14-6812	Q	Churchard	F	73	White	0
13	393-38-9502	N	Corpes	F	16	White	0
14	288-05-6370	R	Brixey	F	16	Native American/Alaska Native	0
15	784-54-9931	M	Goudie	M	46	Pacific Islander	0
16	662-21-6522	G	Stanlack	M	69	White	1
17	628-73-1801	C	McMurty	M	37	Declined to Identify	0
18	370-19-2271	I	Scothorn	F	50	Asian	0
19	458-98-8860	J	Helgass	M	37	Declined to Identify	0
20	728-31-2493	W	Chittock	F	70	Asian	0
21	823-34-5523	F	Prendergast	M	55	Asian	0
22	621-70-7472	T	Bissiker	F	63	Native American/Alaska Native	0
23	344-36-7156	M	Mandell	F	44	Asian	1
24	455-21-3671	D	Coste	F	11	Declined to Identify	0
25	259-10-0339	Q	Dodridge	M	8	Asian	0
26	720-54-2625	C	Pavie	F	4	Native American/Alaska Native	1
27	661-92-7059	Z	Sleightholm	M	42	African American	0
28	598-53-3927	S	Noads	F	68	African American	0
29	715-74-5338	J	Filkin	M	22	Pacific Islander	0
30	669-74-2146	C	Bilby	F	58	Declined to Identify	0

Patient_Source

Figure 2.1: Patient Source CSV File

Source 2 – ER Visit Source (CSV)

- File Name: HospitalER_Data.csv
- Contains emergency room visit transactional data
- Attributes include:
  - Visit_ID
  - Patient_ID
  - Admission_Date
  - Admission_Flag
  - Wait_Time
  - Satisfaction_Score
  - Department reference field

	A	B	C	D	E	F	G
1	Visit_ID	Patient_ID	Admission_Date	Department_Referral	Admission_Flag	Satisfaction_Score	Wait_Time
2	V0001	145-39-5406	20-03-2024 08:47	None	FALSE	10	39
3	V0002	316-34-3057	15-06-2024 11:29	None	TRUE		27
4	V0003	897-46-3852	20-06-2024 09:13	General Practice	TRUE	9	55
5	V0004	358-31-9711	2024-04-02 22:34	General Practice	TRUE	8	31
6	V0005	289-26-0537	2024-04-09 17:48	Orthopedics	FALSE		10
7	V0006	255-51-2877	20-04-2023 00:13	None	FALSE		59
8	V0007	465-97-0990	23-08-2023 08:26	None	TRUE		43
9	V0008	157-31-7520	29-07-2023 16:57	None	TRUE		23
10	V0009	432-34-5614	19-02-2024 06:54	None	FALSE	1	42
11	V0010	609-17-8678	2024-11-10 05:25	None	FALSE		51
12	V0011	497-14-6812	26-07-2024 01:45	Gastroenterology	TRUE		34
13	V0012	393-38-9502	2024-10-03 22:02	Orthopedics	FALSE		39
14	V0013	288-05-6370	2023-12-11 16:00	General Practice	TRUE		53
15	V0014	784-54-9931	25-06-2023 09:40	None	FALSE		45
16	V0015	662-21-6522	2023-04-05 13:16	None	TRUE		49
17	V0016	628-73-1801	19-09-2023 01:53	None	TRUE		57
18	V0017	370-19-2271	25-05-2024 22:11	General Practice	FALSE		35
19	V0018	458-98-8860	25-06-2023 18:59	None	FALSE		55
20	V0019	728-31-2493	2023-04-09 16:15	Physiotherapy	TRUE		50
21	V0020	823-34-5523	16-11-2023 23:46	None	TRUE	2	40
22	V0021	621-70-7472	30-06-2023 05:22	None	TRUE		25
23	V0022	344-36-7156	22-05-2023 16:48	None	FALSE	2	51
24	V0023	455-21-3671	17-11-2023 07:24	None	FALSE	4	30
25	V0024	259-10-0339	26-01-2024 05:58	General Practice	FALSE		16
26	V0025	720-54-2625	24-05-2023 14:42	None	TRUE		23
27	V0026	661-92-7059	2023-12-04 21:02	None	FALSE	0	51
28	V0027	598-53-3927	2024-04-09 02:30	None	FALSE		58
29	V0028	715-74-5338	16-12-2023 13:02	General Practice	FALSE		25
30	V0029	669-74-2146	2024-11-03 17:06	Orthopedics	FALSE		55

Figure 2.2: ER Visit Source CSV File

Source 3 – Department Lookup (Excel)

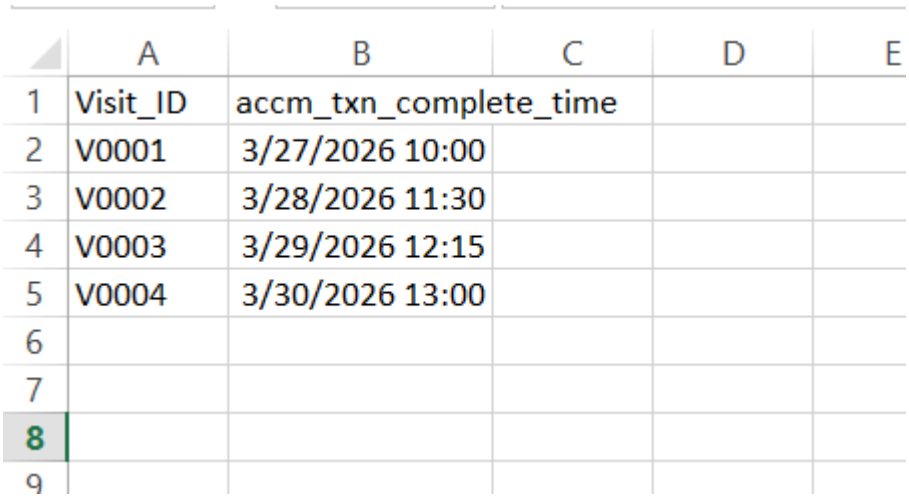
- File Name: Department_Lookup.xlsx
- Contains department reference data
- Attributes include:
  - Department_ID
  - Department_Name
  - Department_Group

	A	B	C
1	Department_ID	Department_Name	Department_Group
2	D001	None	Unknown
3	D002	General Practice	General
4	D003	Orthopedics	Surgical
5	D004	Gastroenterology	Medical
6	D005	Physiotherapy	Support
7	D006	Neurology	Medical
8	D007	Cardiology	Medical
9	D008	Renal	Medical
10			
11			
12			
13			
14			

Figure 2.3: Department Lookup Excel File

Source 4 – Completion Data (Excel) *(Used in Task 6)*

- File Name: ER_Visit_Completion.xlsx
- Contains completion timestamps for visits
- Attributes include:
  - Visit_ID
  - accm_txn_complete_time



	A	B	C	D	E
1	Visit_ID	accm_txn_complete_time			
2	V0001	3/27/2026 10:00			
3	V0002	3/28/2026 11:30			
4	V0003	3/29/2026 12:15			
5	V0004	3/30/2026 13:00			
6					
7					
8					
9					

Figure 2.4: Completion Update Excel File

Therefore, this project uses two main source types: CSV and Excel.This source is specifically used for the accumulating fact update process in Task 6.

Important Note on Source Structure

The source column names are kept in their original extracted format to maintain consistency with the ETL mappings and staging design used in later tasks.

Although some column names are not fully standardized, this reflects real-world data warehousing scenarios where raw source data is preserved, and transformations are applied during the ETL process and data warehouse design.

2.3 Reason for Using Multiple Sources

The dataset was intentionally divided into multiple source formats to simulate a real-world data integration scenario. In practical environments:

- transactional data may come from operational systems (CSV exports),
- master/reference data may come from spreadsheets (Excel),
- and additional process data may come from external systems.

Using multiple sources provides the following benefits:

- demonstrates ETL capability,
- enables data integration from heterogeneous systems,
- supports staging design,
- and satisfies assignment requirements.

Source-to-Entity Mapping

Each source file contributes to different entities in the system:

Source File	Entity	Description
Patient_Source.csv	PATIENT	Contains descriptive patient data
ER_Visit_Source.csv	ER_VISIT	Contains transactional visit data
Department_Lookup.xlsx	DEPARTMENT	Contains department reference data
ER_Visit_Completion.xlsx	ER_VISIT(Update)	Contains completion time for accumulating fact

2.4 Data Preparation Steps

Before ETL implementation, the source files were reviewed to identify missing values, inconsistent formats, and key relationships. Basic checks were performed on data types and attribute consistency, while detailed cleaning and transformation were handled during the SSIS ETL process.

2.5 Need for Staging Area

Based on the practical sessions, data from multiple sources should first be loaded into a **staging area** before being transformed into the data warehouse.

The staging area is used to:

- store raw extracted data,
- isolate source systems from the warehouse,
- perform initial cleaning and transformation,
- and simplify debugging and reprocessing.

In this project, staging tables such as:

- Stg_Patient
- Stg_ER_Visit
- Stg_Department

will be used to temporarily store data before loading it into dimension and fact tables.

2.6 Suitability of Data Sources for ETL

The prepared data sources are suitable for ETL because:

- they represent different formats (CSV and Excel),
- they are logically separated by entity,
- they support integration through common keys,
- and they align with the requirements of dimensional modeling.

This setup enables the implementation of a structured ETL pipeline using SSIS in later tasks

2.7 Conclusion

In Task 2, the Hospital ER dataset was successfully prepared into multiple data source formats, including CSV and Excel files. Each source was mapped to the appropriate entity and prepared for extraction.

The use of multiple heterogeneous sources reflects real-world data integration scenarios and provides a strong foundation for the ETL process. The prepared sources will be loaded into a staging area in the next task, enabling transformation and integration into the data warehouse.

TASK 3- High-Level DWBI Architecture

3.1 Introduction

Task 3 focuses on designing a **high-level Data Warehousing and Business Intelligence (DWBI) architecture** for the selected Hospital Emergency Room dataset.

In data warehousing systems, architecture plays a critical role in defining how data flows from source systems to the data warehouse and finally to reporting and

analysis tools. The architecture ensures proper data integration, transformation, storage, and accessibility.

The proposed architecture follows a layered approach consisting of:

- Source Layer
- Staging Layer
- ETL Layer
- Data Warehouse Layer
- Reporting Layer

3.2 Overview of the Architecture

The high-level architecture illustrates how data flows from multiple heterogeneous source systems into a centralized data warehouse for analytical processing and reporting.

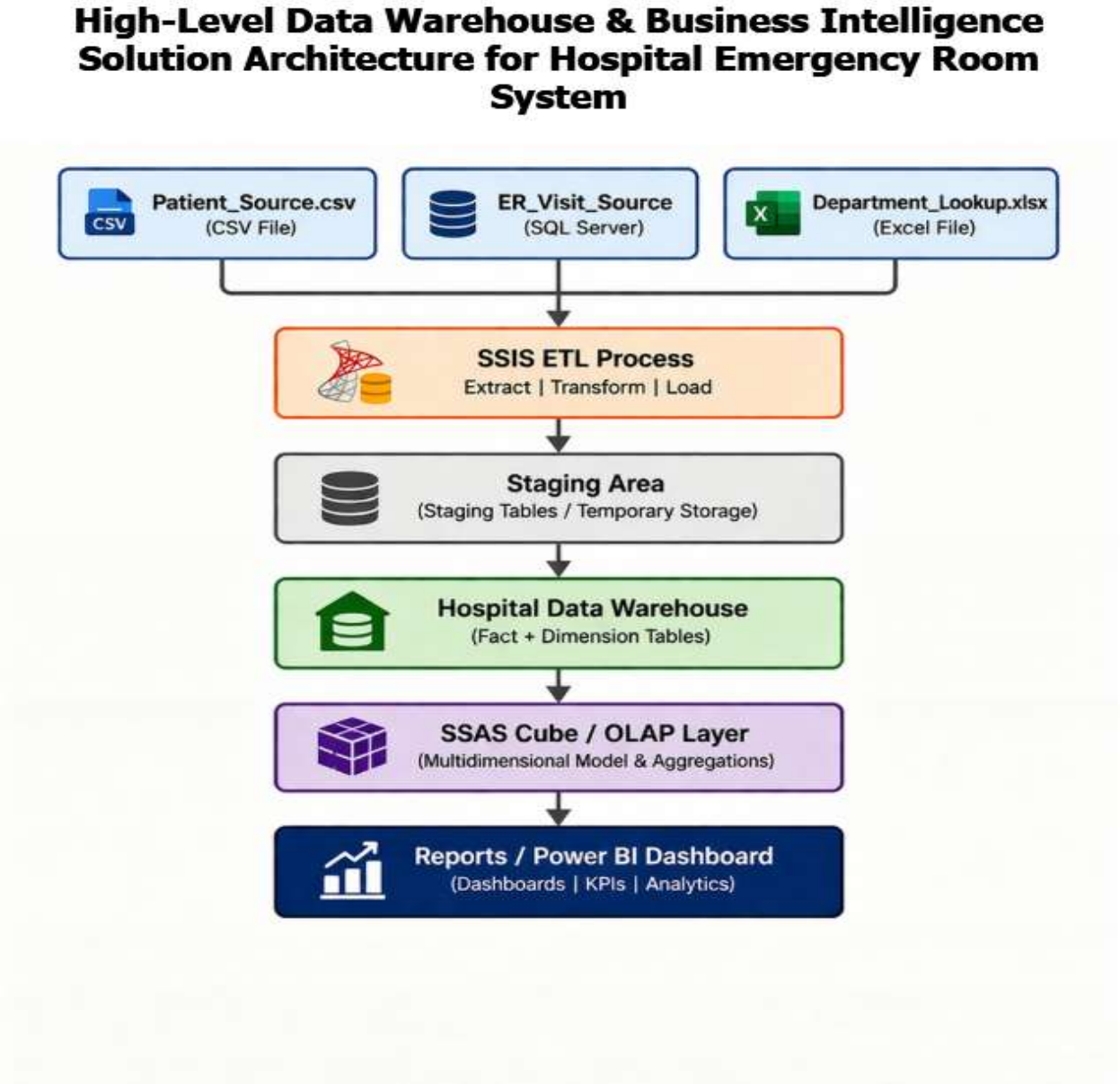


Figure 3.1:High-level DWBI Architecture for Hospital ER Dataset

3.3 Source Layer

The source layer consists of the original data files used in this project. These sources represent raw operational data collected from different systems.

The following source types are used:

- CSV files:
  - Patient_Source.csv
  - HospitalER_Data.csv
- Excel files:
  - Department_Lookup.xlsx
  - ER_Visit_Completion.xlsx

These sources contain transactional, reference, and process-related data required for building the data warehouse.

3.4 Staging Layer

The staging layer acts as an intermediate storage area where raw data from source systems is temporarily stored before transformation.

In this project, staging tables are created to store extracted data in a structured format. These include:

- Stg_Patient
- Stg_ER_Visit
- Stg_Department

The staging layer provides the following benefits:

- isolates source systems from the data warehouse
- allows data cleaning and validation
- simplifies debugging and reprocessing
- supports integration from multiple sources

Data is first loaded into staging tables before being transformed and loaded into the data warehouse.

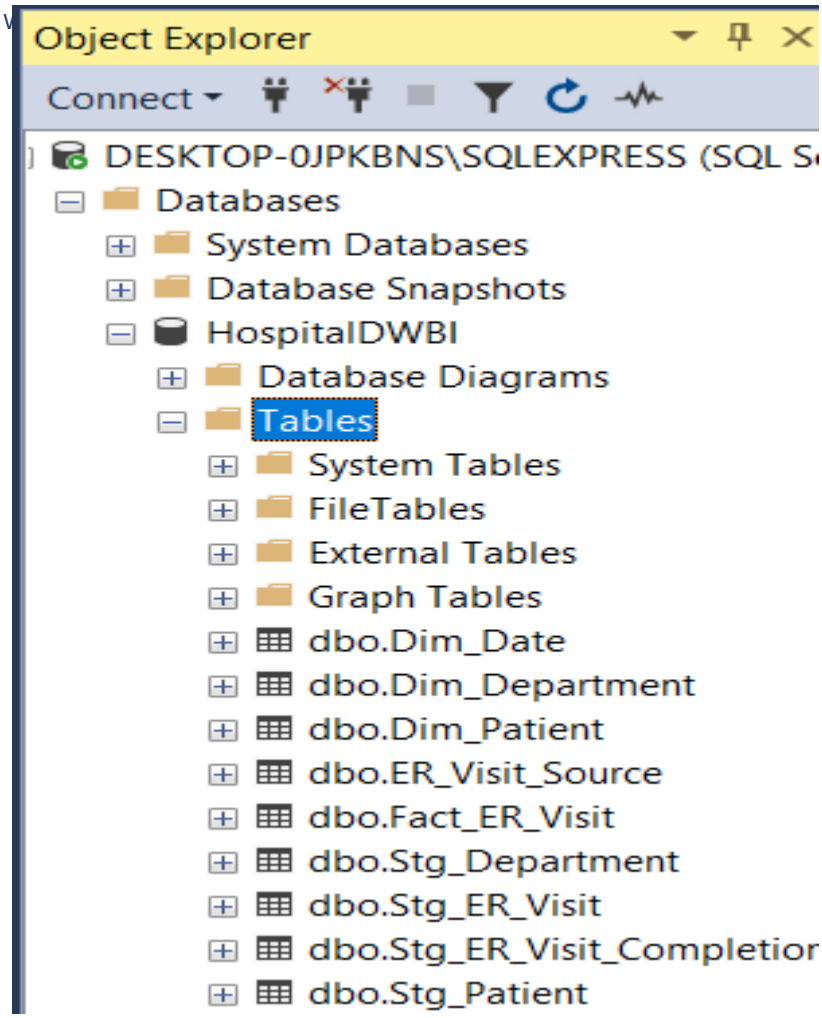


Figure 3.2:Staging tables in SQL Server

3.5 ETL Layer (SSIS)

The ETL (Extract, Transform, Load) layer is responsible for moving data from source systems to the data warehouse.

In this project, ETL is implemented using **SQL Server Integration Services (SSIS)**.

The ETL process consists of:

Extraction

- Data is extracted from CSV and Excel files

Transformation

- Data cleaning
- Handling missing values
- Converting data types
- Applying business rules
- Handling Slowly Changing Dimensions (SCD)

Loading

- Loading dimension tables first
- Loading fact table after dimensions

The ETL workflow follows this sequence:

1. Source → Staging
2. Staging → Dimensions
3. Dimensions → Fact Table

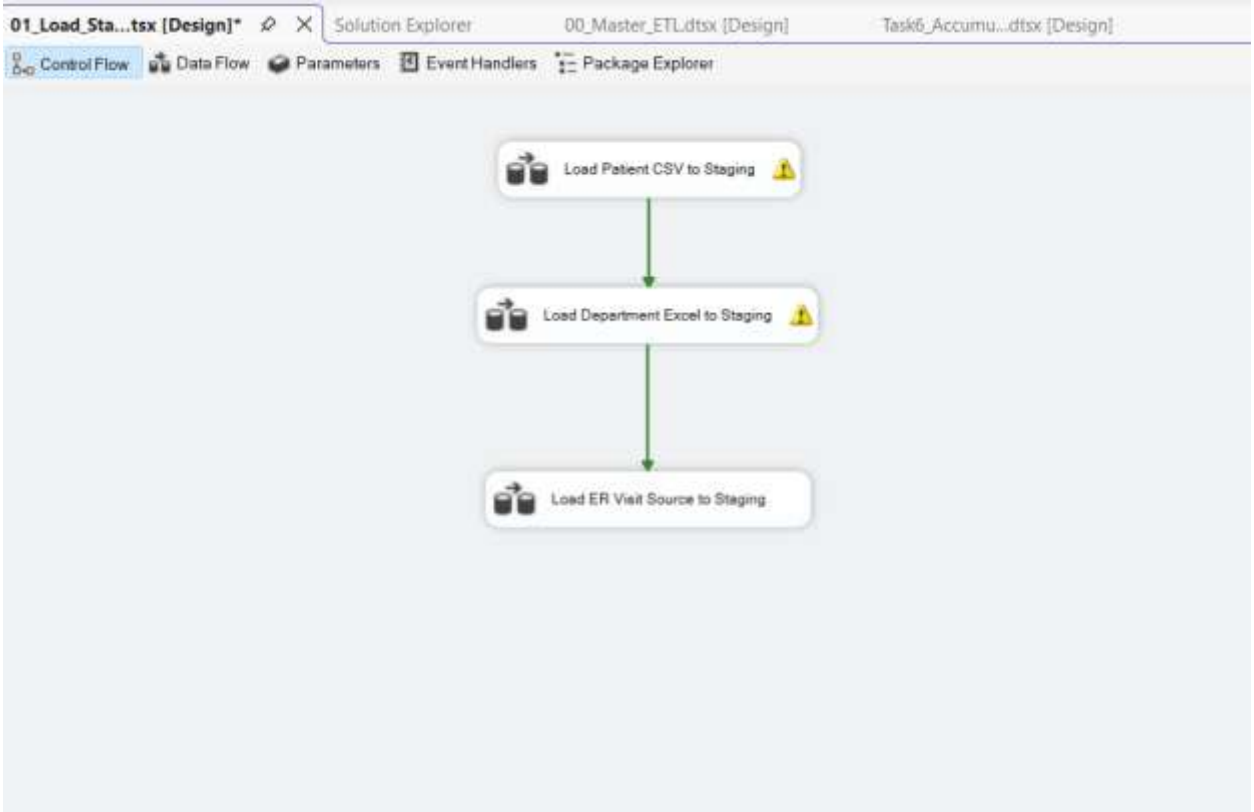


Figure 3.3:SSIS Control flow and ETL Process

Figure 3.4:Data Flow showing extraction,transformation and loading

Data Conversion transformation is used to convert string values into appropriate numeric data types before loading into the staging tables.

3.6 Data Warehouse Layer

The data warehouse layer stores cleaned, integrated, and structured data using a dimensional model.

The following tables are created:

Dimension Tables

- Dim_Patient

- Dim_Department (Slowly Changing Dimension Type 2)
- Dim_Date

Fact Table

- Fact_ER_Visit

The fact table stores measurable data such as:

- Wait_Time
- Satisfaction_Score
- Visit_Count

Dimension tables provide descriptive context for analysis.

The design follows a **star schema**, where the fact table is connected to multiple dimensions.

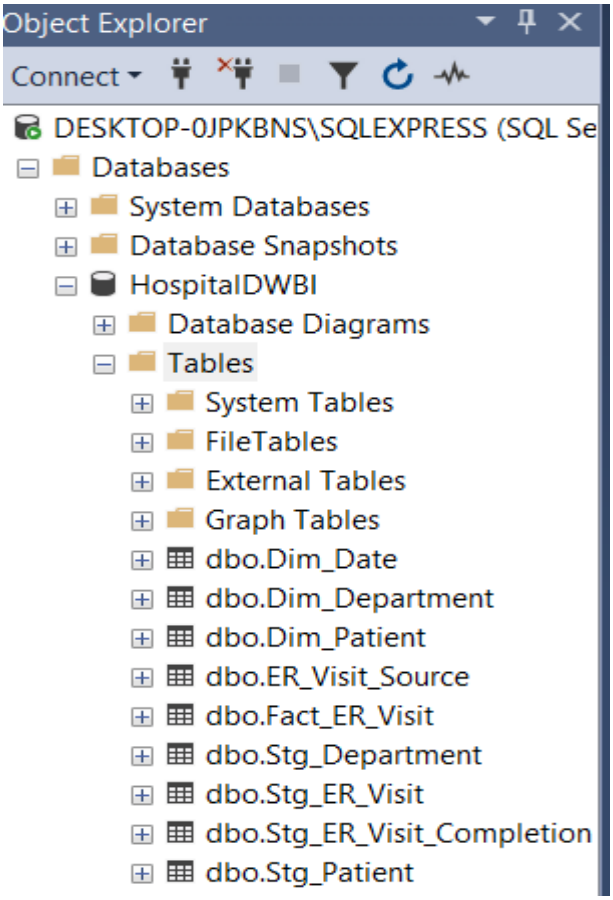


Figure 3.5:Data Warehouse tables(fact and dimension tables)

3.7 Reporting Layer

The reporting layer enables users to perform analytical queries and generate insights such as patient flow analysis, department performance, and satisfaction trends. This layer supports decision-making by providing aggregated and meaningful information from the data warehouse.

This layer supports:

- query analysis
- dashboards
- reports
- performance evaluation

Using the warehouse, users can analyze:

- average waiting time by department
- patient demographics
- satisfaction trends
- visit patterns over time

This layer provides meaningful insights for decision-making

3.8 Data Flow Explanation

The data flow in the architecture follows a structured pipeline:

1. Data is collected from multiple source systems (CSV and Excel)
2. Data is extracted and loaded into staging tables
3. Data is transformed using ETL processes
4. Cleaned data is loaded into dimension tables
5. Fact table is populated using surrogate keys
6. Data warehouse is used for reporting and analysis

This structured flow ensures data consistency, integrity, and scalability.

Conclusion

In Task 3, a high-level DWBI architecture was designed for the Hospital Emergency Room dataset. The architecture clearly defines how data flows from source systems through staging and ETL processes into the data warehouse and finally to reporting.

The layered approach ensures proper data integration, transformation, and storage, making it suitable for analytical processing. This architecture provides a strong foundation for implementing the data warehouse and ETL processes in subsequent tasks.

TASK 4-Data Warehouse Design

4.1 Introduction

Task 4 focuses on designing the data warehouse schema for the Hospital Emergency Room dataset. The objective is to transform the conceptual ER model into a dimensional model suitable for analytical processing.

A **star schema** design is used, where a central fact table is connected to multiple dimension tables. This structure supports efficient querying, reporting, and business intelligence analysis

4.2 Overview of Star Schema

The star schema consists of:

- One **Fact Table** (central table storing measures)
- Multiple **Dimension Tables** (descriptive attributes)

In this project:

- Fact Table: `Fact_ER_Visit`
- Dimension Tables:
  - `Dim_Patient`
  - `Dim_Department`
  - `Dim_Date`

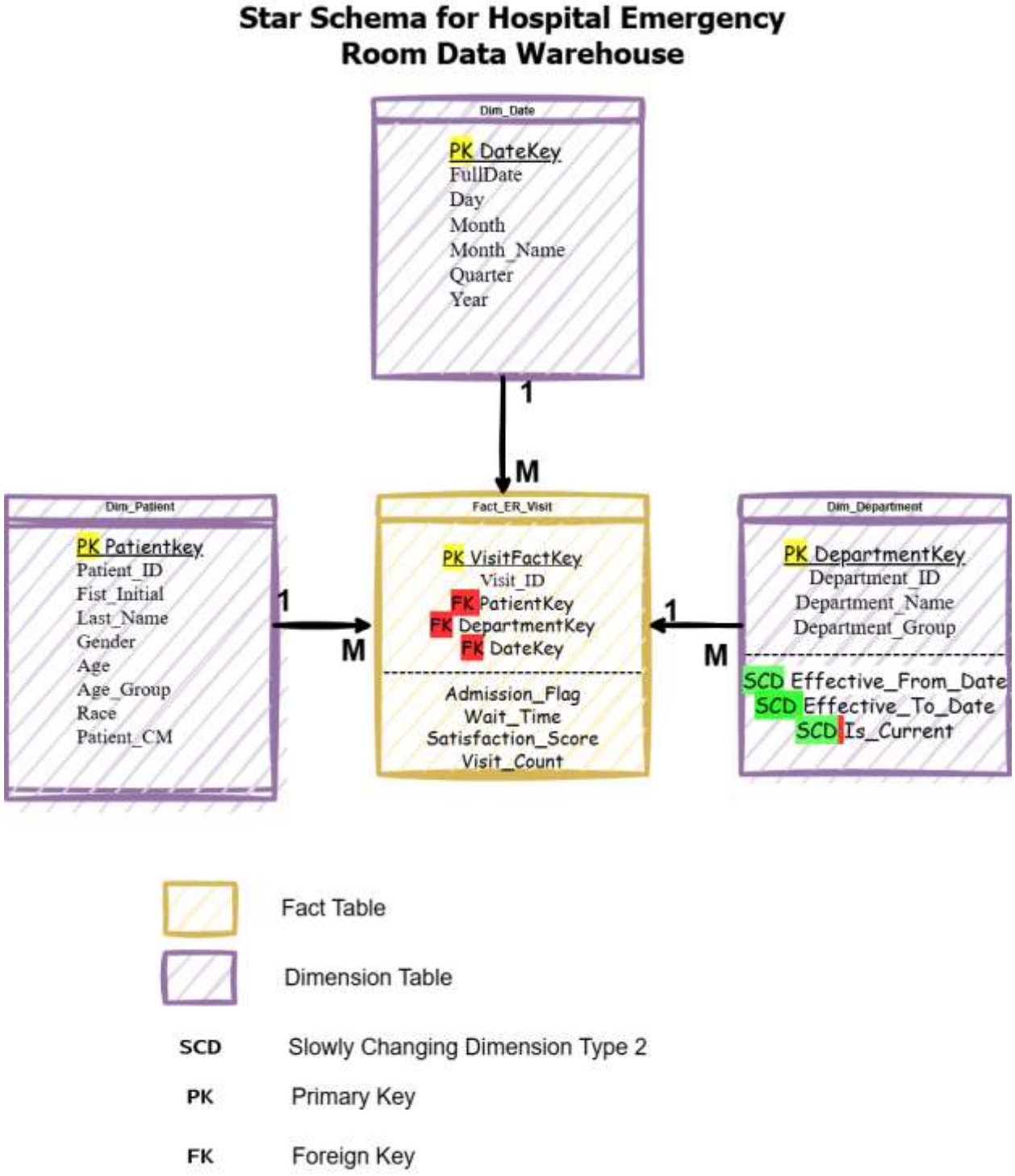


Figure 4.1:Star Schema for data warehouse

4.3 Fact table Design and Implementation

The fact table captures the core business event of emergency room visits.

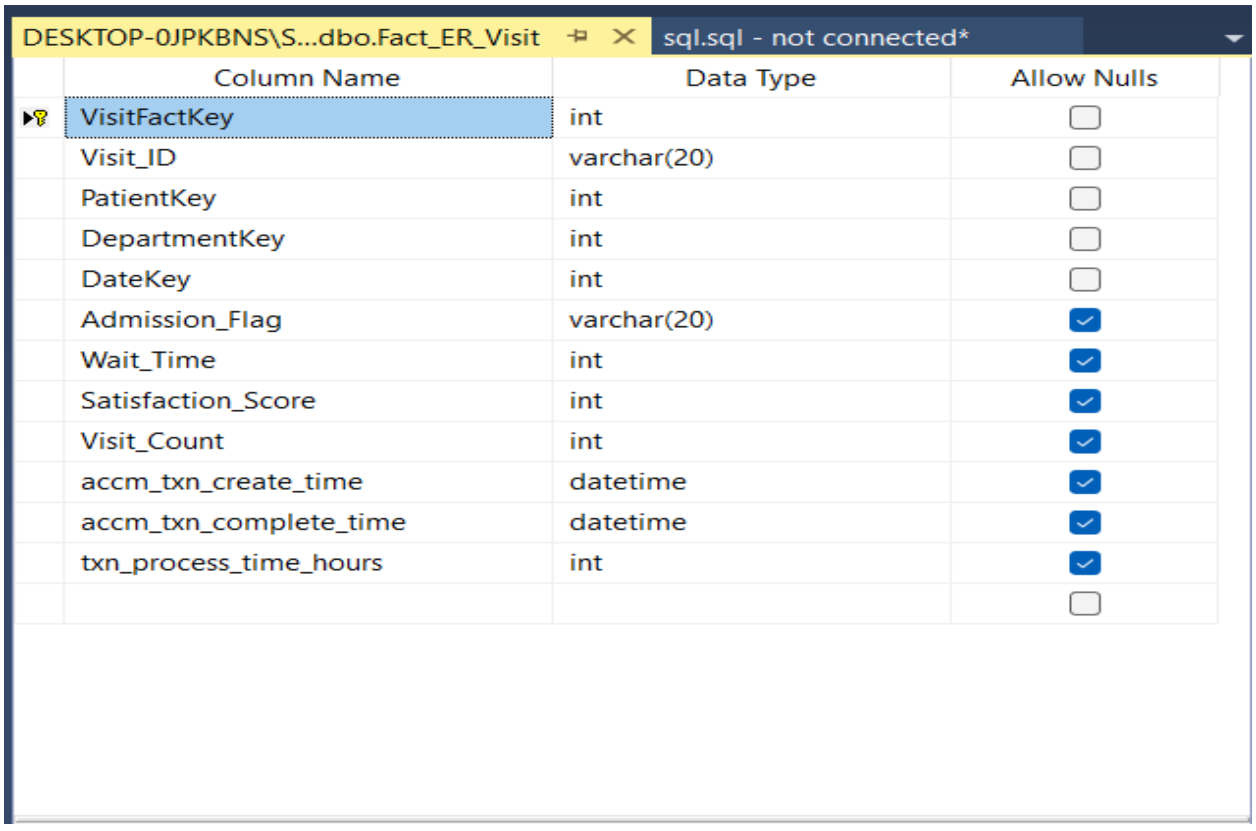
One record represents one ER visit

Attributes

- VisitFactKey (Primary Key)
- Visit_ID
- PatientKey (Foreign Key)
- DepartmentKey (Foreign Key)
- DateKey (Foreign Key)
- Admission_Flag
- Wait_Time
- Satisfaction_Score
- Visit_Count

Measures

- Wait_Time
- Satisfaction_Score
- Visit_Count



Column Name	Data Type	Allow Nulls
VisitFactKey	int	☐
Visit_ID	varchar(20)	☐
PatientKey	int	☐
DepartmentKey	int	☐
DateKey	int	☐
Admission_Flag	varchar(20)	☑
Wait_Time	int	☑
Satisfaction_Score	int	☑
Visit_Count	int	☑
accm_txn_create_time	datetime	☑
accm_txn_complete_time	datetime	☑
txn_process_time_hours	int	☑
		☐

Figure 4.2:Fact ER Visit table structure in SQL Server

98 %

Results Messages

	VisitFactKey	Visit_ID	PatientKey	DepartmentKey	DateKey	Admission_Flag	Wait_Time	Satisfaction_Score	Visit_Count	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	1	V0001	494	17	20240320	FALSE	39	10	1	2026-03-25 01:31:37.007	2026-03-25 10:00:00.000	9
2	2	V0002	2482	17	20240615	TRUE	27	NULL	1	2026-03-25 01:31:37.007	2026-03-25 11:30:00.000	10
3	3	V0003	9186	18	20240620	TRUE	55	9	1	2026-03-25 01:31:37.007	2026-03-25 12:15:00.000	11
4	4	V0004	2946	18	20240204	TRUE	31	8	1	2026-03-25 01:31:37.007	2026-03-25 13:00:00.000	12
5	5	V0005	2171	19	20240904	FALSE	10	NULL	1	2026-03-25 01:31:37.007	NULL	NULL
6	6	V0006	1788	17	20230420	FALSE	59	NULL	1	2026-03-25 01:31:37.007	NULL	NULL
7	7	V0007	4206	17	20230823	TRUE	43	NULL	1	2026-03-25 01:31:37.007	NULL	NULL
8	8	V0008	620	17	20230729	TRUE	23	NULL	1	2026-03-25 01:31:37.007	NULL	NULL
9	9	V0009	3818	17	20240219	FALSE	42	1	1	2026-03-25 01:31:37.007	NULL	NULL
10	10	V0010	5790	17	20241011	FALSE	51	NULL	1	2026-03-25 01:31:37.007	NULL	NULL
11	11	V0011	4548	20	20240726	TRUE	34	NULL	1	2026-03-25 01:31:37.007	NULL	NULL
12	12	V0012	3353	19	20240310	FALSE	39	NULL	1	2026-03-25 01:31:37.007	NULL	NULL
13	13	V0013	2160	18	20231112	TRUE	53	NULL	1	2026-03-25 01:31:37.007	NULL	NULL
14	14	V0014	7887	17	20230625	FALSE	45	NULL	1	2026-03-25 01:31:37.007	NULL	NULL
15	15	V0015	6416	17	20230504	TRUE	49	NULL	1	2026-03-25 01:31:37.007	NULL	NULL
16	16	V0016	6014	17	20230919	TRUE	57	NULL	1	2026-03-25 01:31:37.007	NULL	NULL

Figure 4.3:Sample data in Fact ER Visit Table

4.4 Dimension Tables Design and Implementation

4.4.1 Dim_Patient

Stores descriptive information about patients.

Attributes

- PatientKey (Surrogate Key)
- Patient_ID
- First_Initial
- Last_Name
- Gender
- Age
- Age_Group
- Race
- Patient_CM

DESKTOP-0JPKBNS\...- dbo.Dim_Patient

Column Name	Data Type	Allow Nulls
PatientKey	int	☐
Patient_ID	varchar(20)	☐
First_Initial	varchar(10)	☑
Last_Name	varchar(50)	☑
Gender	varchar(20)	☑
Age	int	☑
Age_Group	varchar(20)	☑
Race	varchar(50)	☑
Patient_CM	varchar(50)	☑
		☐

Figure 4.4:Dim Patient table design or data

4.4.2 Dim_Department

Attributes

- DepartmentKey (Surrogate Key)
- Department_ID
- Department_Name
- Department_Group
- Effective_From_Date
- Effective_To_Date
- Is_Current

SCD Type 2 Explanation

The Slowly Changing Dimension Type 2 technique is used to maintain historical data.

- When a change occurs:
  - Old record is expired (`Is_Current = 0`)
  - New record is inserted

- Effective dates are updated

This allows tracking of changes over time

DESKTOP-0JPKBNS...o.Dim_Department

Column Name	Data Type	Allow Nulls
DepartmentKey	int	☐
Department_ID	varchar(20)	☐
Department_Name	varchar(100)	☑
Department_Group	varchar(50)	☑
Effective_From_Date	date	☑
Effective_To_Date	date	☑
Is_Current	bit	☑
		☐

Figure 4.5: Dim Department table showing multiple records

The Slowly Changing Dimension Type 2 (SCD Type 2) was implemented using the SSIS Slowly Changing Dimension transformation.

The SSIS SCD component automatically handles:

- Identifying changes in dimension attributes
- Expiring old records (setting Is_Current = 0)
- Inserting new records with updated values
- Managing Effective_From_Date and Effective_To_Date

This approach ensures accurate historical tracking and follows standard ETL practices used in real-world data warehouse systems.

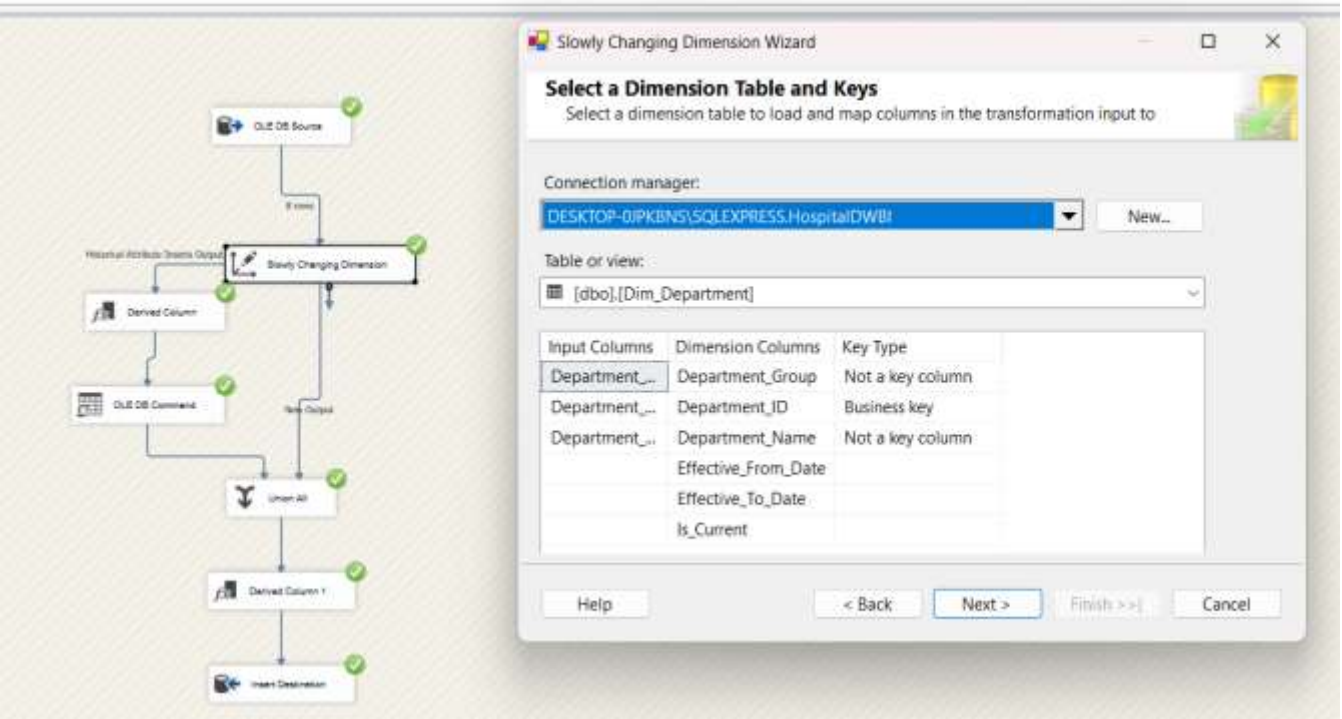


Figure 4.6: SCD Transformation Configuration in SSIS

4.4.3 Dim_Date

Stores date-related attributes for time-based analysis.

Attributes

- DateKey
- FullDate
- Day
- Month
- Month_Name
- Quarter
- Year

Column Name	Data Type	Allow Nulls
DateKey	int	☐
FullDate	date	☐
Day	int	☑
Month	int	☑
Month_Name	varchar(20)	☑
Quarter	int	☑
Year	int	☑
		☐

Figure 4.7 :Dim Data table

4.5 Relationships in Star Schema

The relationships are defined as:

- Fact_ER_Visit → Dim_Patient
- Fact_ER_Visit → Dim_Department
- Fact_ER_Visit → Dim_Date

Each record in the fact table references dimension tables using foreign keys.

This structure enables efficient querying and simplifies data analysis.

4.6 Conclusion

In Task 4, a star schema was designed and implemented for the Hospital Emergency Room dataset. The schema consists of a central fact table connected to multiple dimension tables.

The use of surrogate keys, SCD Type 2, and well-defined relationships ensures that the data warehouse supports efficient analytical processing and reporting. This design provides a solid foundation for ETL processes and business intelligence applications

TASK 5-ETL Process Implementation Using SSIS

5.1 Introduction

The ETL (Extract, Transform, Load) process is implemented using **SQL Server Integration Services (SSIS)** to move data from multiple source systems into the data warehouse.

The process ensures:

- Data is extracted from different sources
- Data is cleaned and transformed
- Data is loaded into dimension and fact tables
- Data integrity and consistency are maintained

5.2 ETL Architecture Overview

The ETL process follows this workflow:

Sources → Staging Tables → Dimension Tables → Fact Table

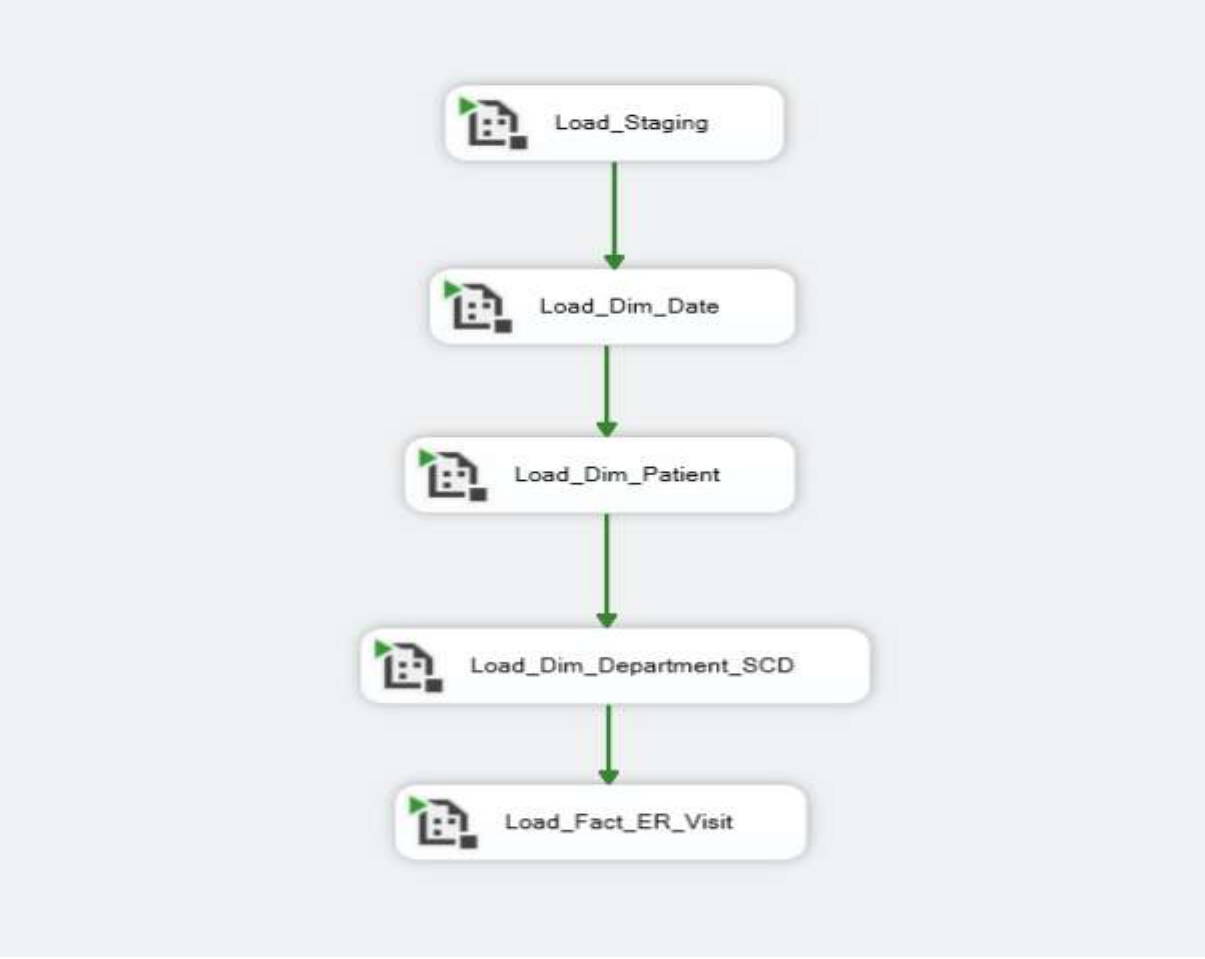


Figure 5.1: Master SSIS Control Flow for ETL Process

5.3 Data Extraction

Data is extracted from multiple sources:

- CSV files (Patient, ER Visit)
- Excel files (Department, Completion data)

The extracted data is loaded into staging tables:

- Stg_Patient
- Stg_Department
- Stg_ER_Visit

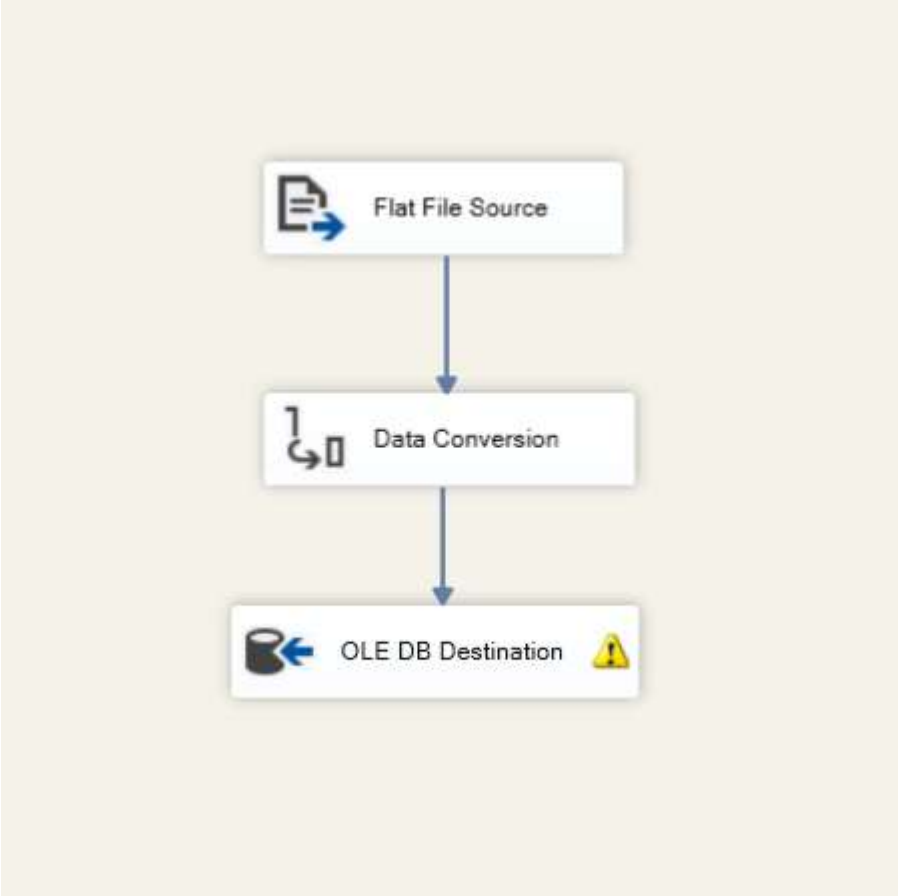
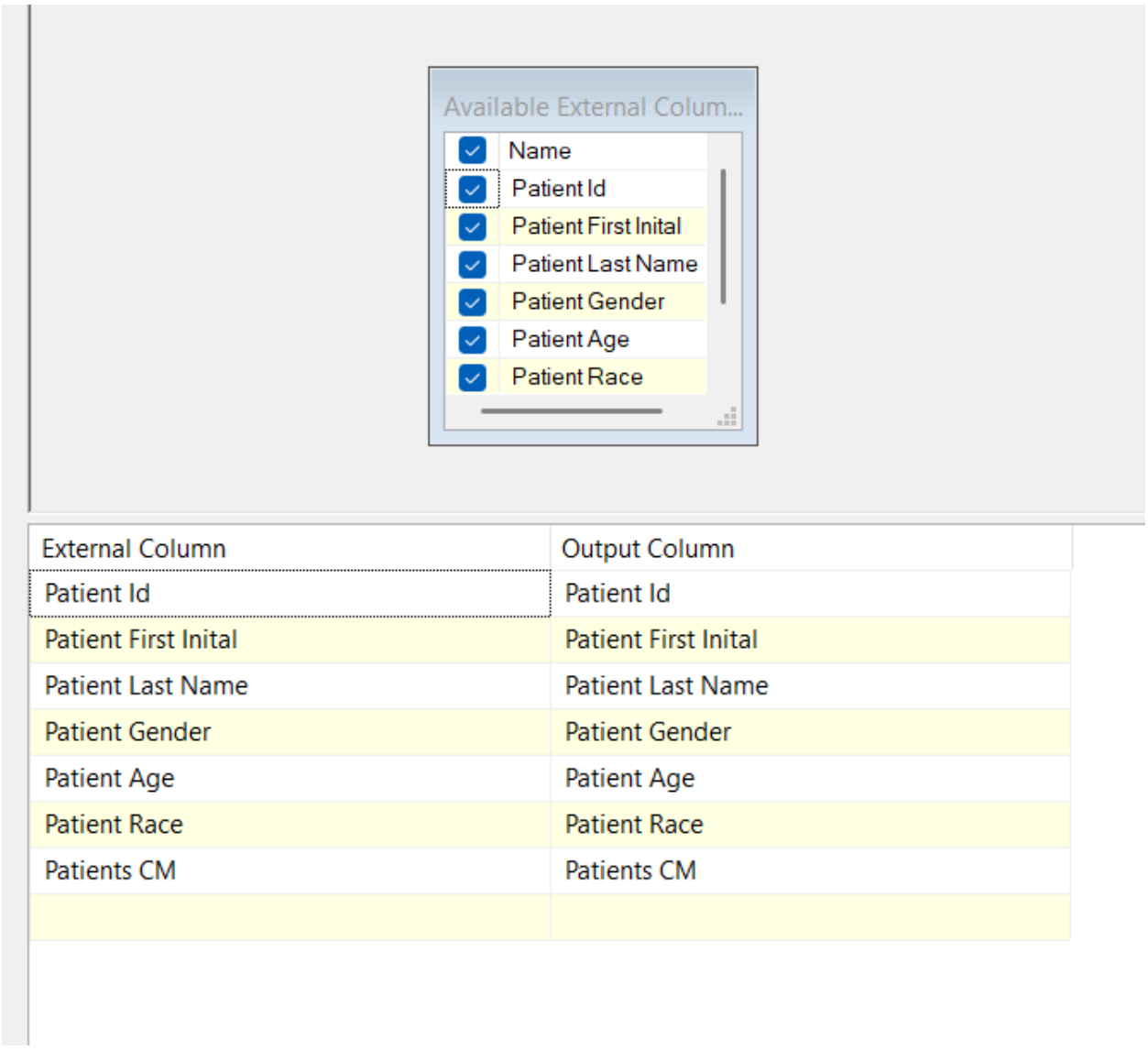
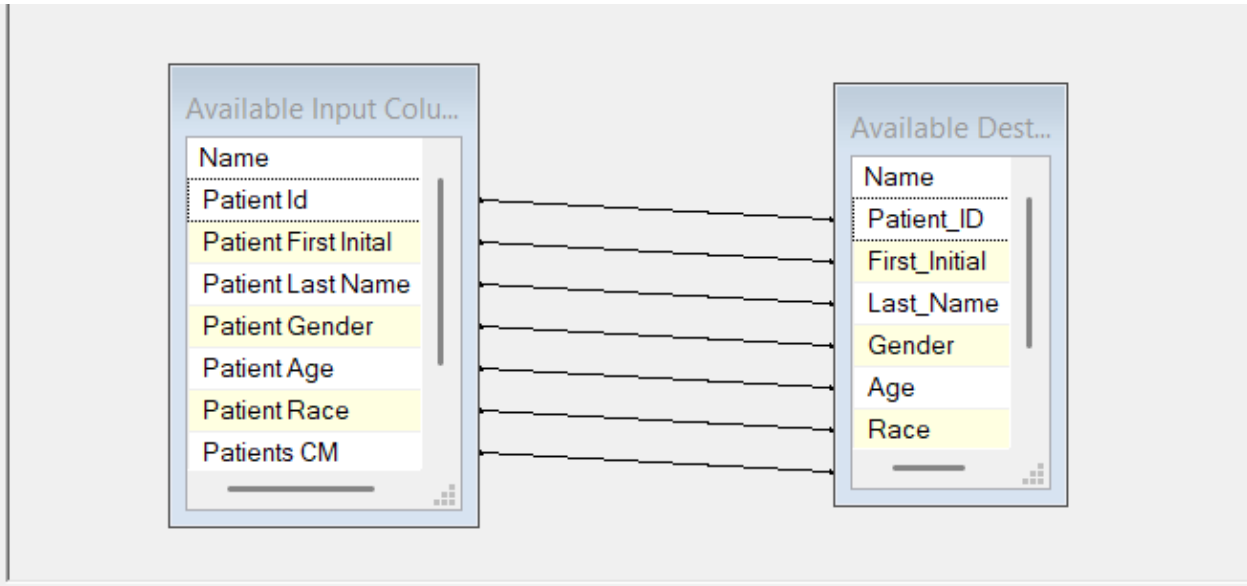


Figure 5.2: Flat File Source for Patient CSV Extraction



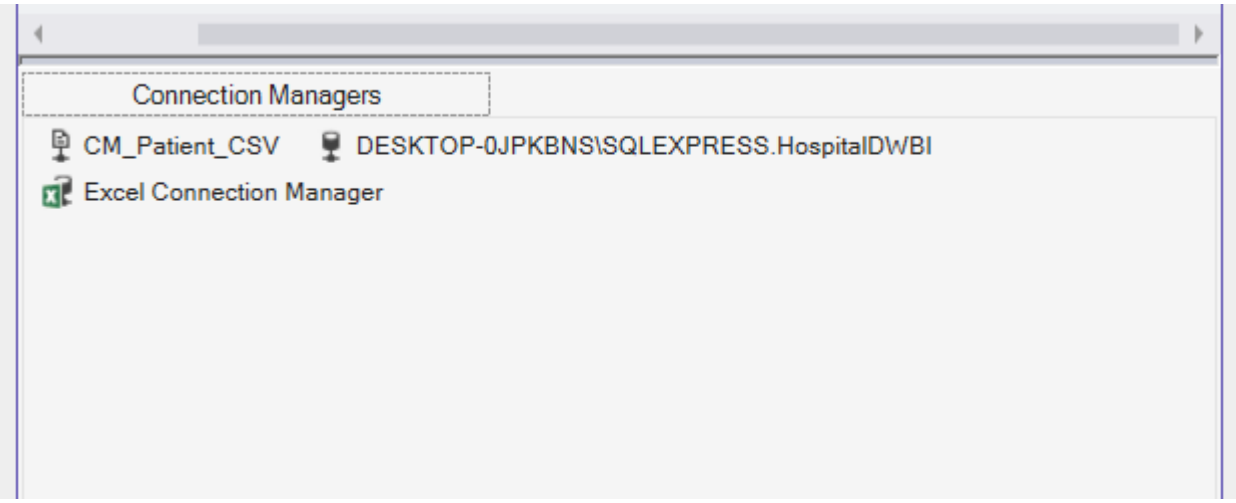
External Column	Output Column
Patient Id	Patient Id
Patient First Inital	Patient First Inital
Patient Last Name	Patient Last Name
Patient Gender	Patient Gender
Patient Age	Patient Age
Patient Race	Patient Race
Patients CM	Patients CM

Figure 5.3: Excel Source for Department Data Extraction



Input Column	Destination Column
Patient Id	Patient_ID
Patient First Inital	First_Initial
Patient Last Name	Last_Name
Patient Gender	Gender
Patient Age	Age
Patient Race	Race
Patients CM	Patient_CM

Figure 5.4: Excel Source for Department Data Extraction



5.4 Data Transformation

Data transformation is performed to improve data quality and standardization.

Transformations applied:

- Data type conversion
- Removing null values
- Standardizing text (Gender, Race)
- Derived columns (Date attributes)

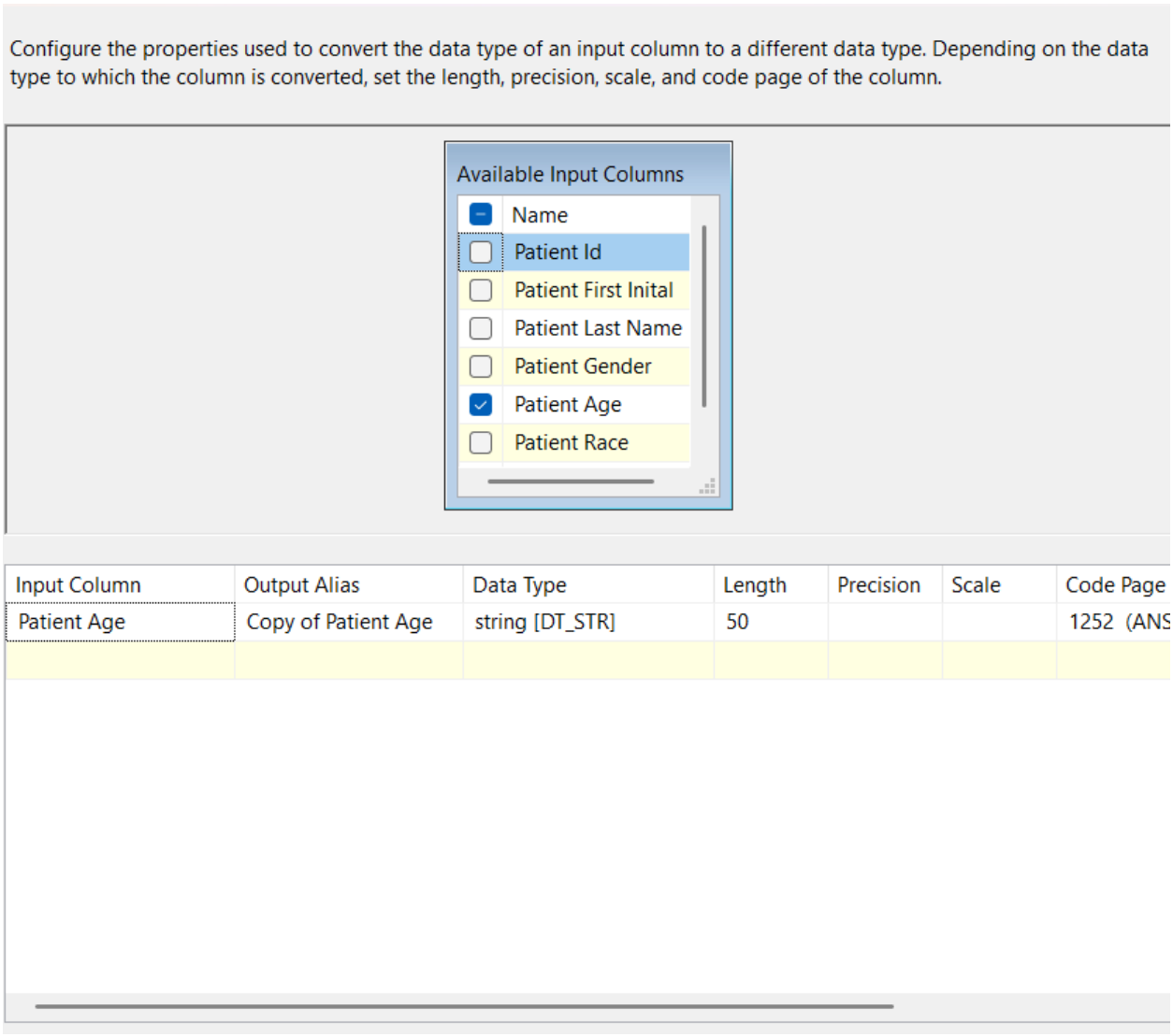


Figure 5.5: Data Conversion Transformation in SSIS

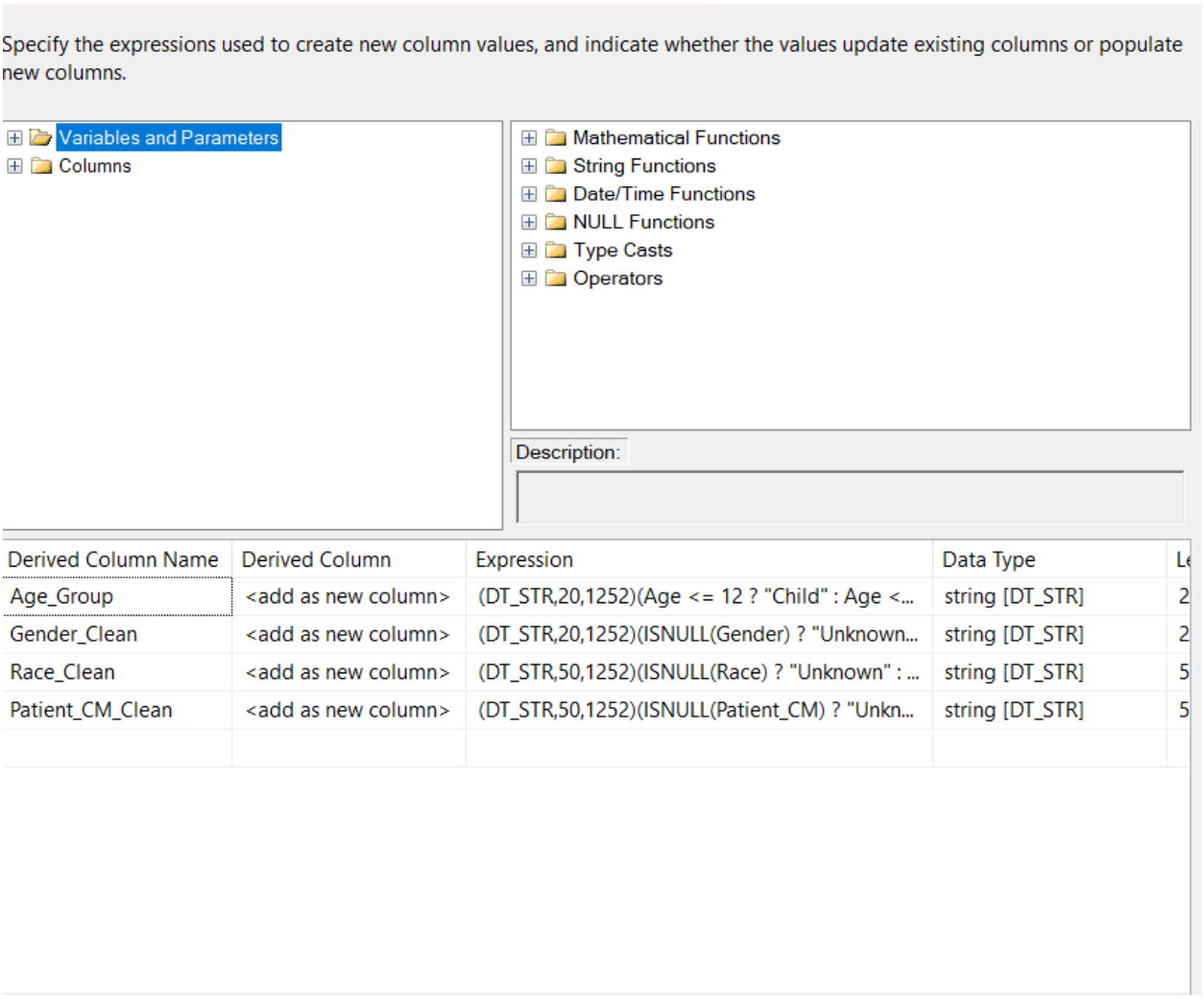


Figure 5.5a: Derived Column Transformation for Age Group or Date Attributes

5.5 Loading Staging Tables

After extraction, data is loaded into staging tables.

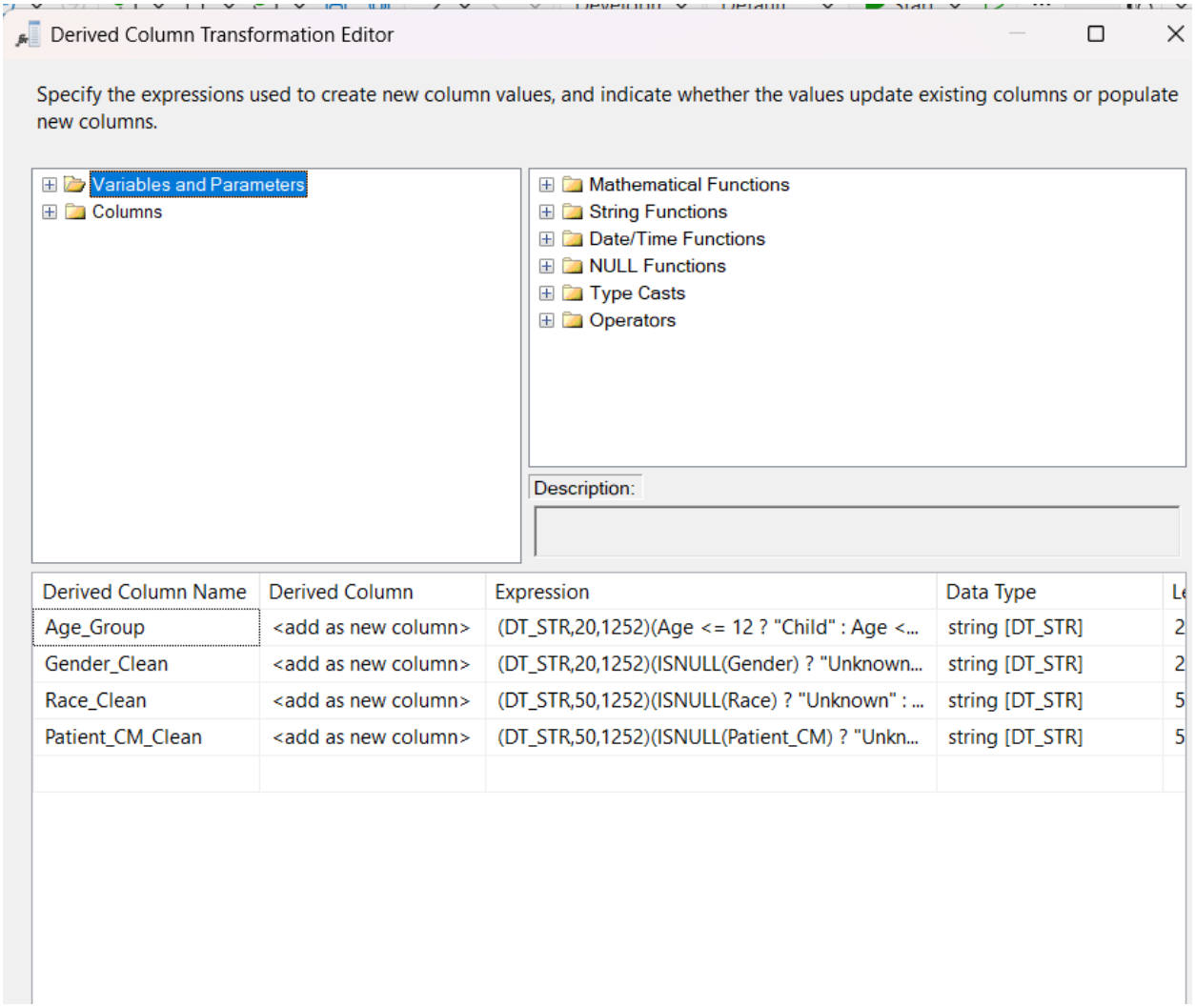


Figure 5.6: OLE DB Destination for Loading Staging Tables

5.6 Dimension Table Loading

Dimension tables are loaded after staging.

Tables loaded:

- Dim_Date
- Dim_Patient
- Dim_Department

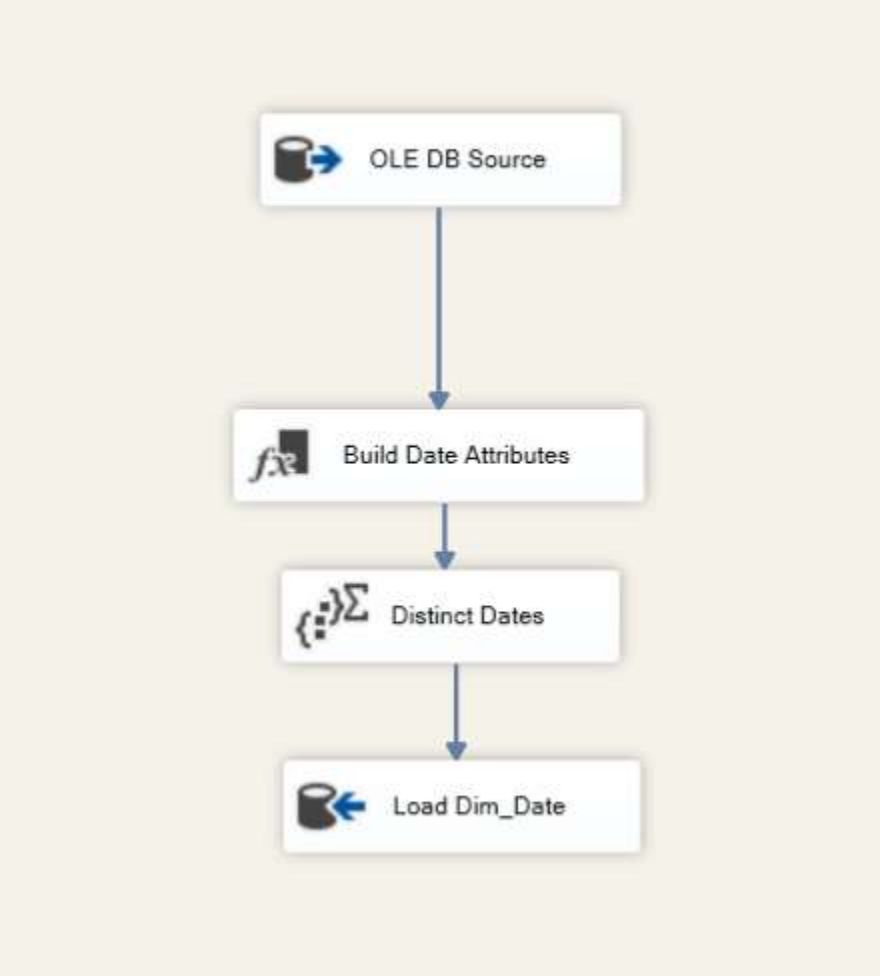


Figure 5.7: Data Flow for Loading Dimension Tables

5.7 Slowly Changing Dimension(SCD Type 2)

SCD Type 2 is implemented using SSIS to track historical changes.

Applied to:

- Dim_Department

Features:

- Detects changes in attributes
- Inserts new record for changes
- Maintains historical data

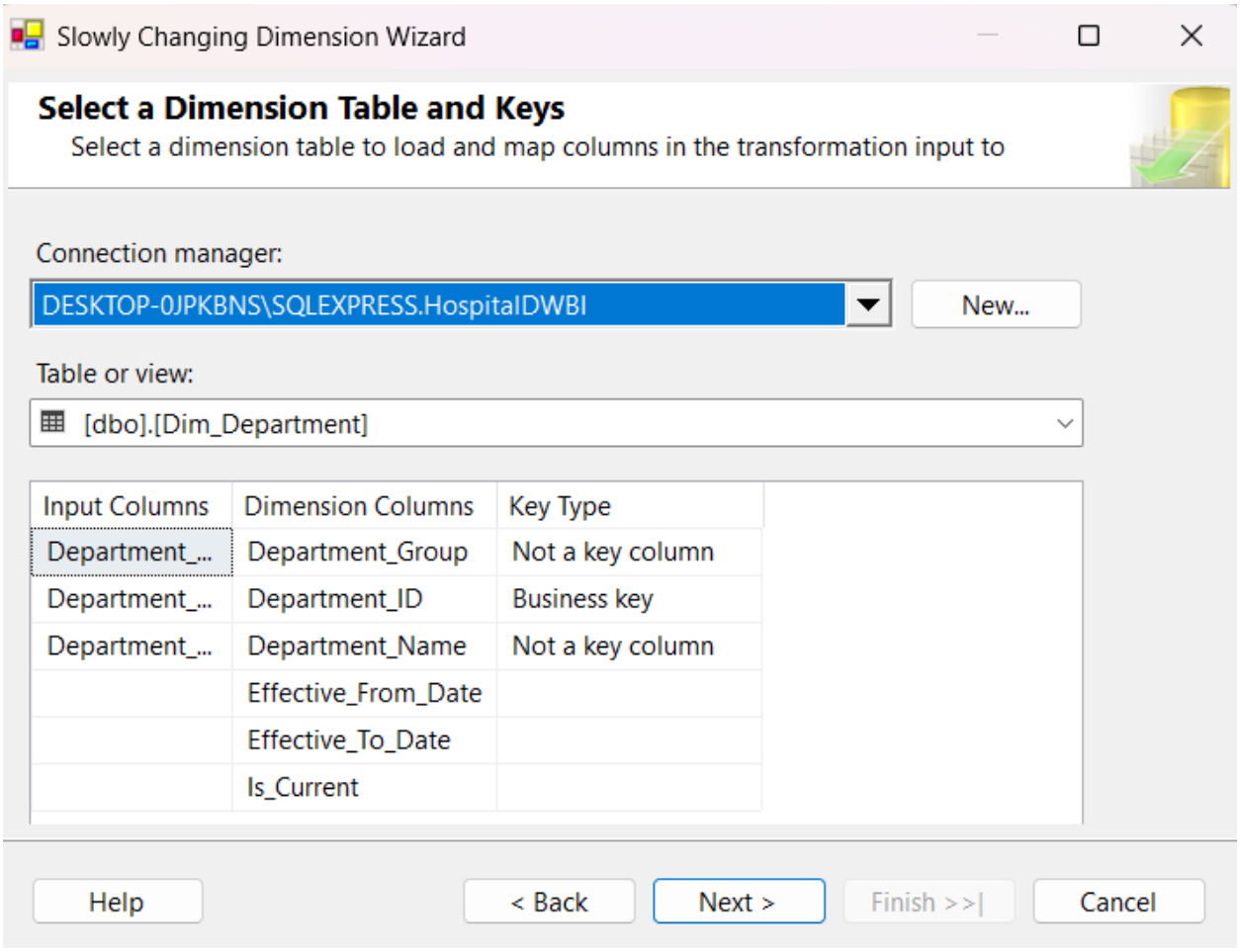


Figure 5.8: Slowly Changing Dimension Transformation in SSIS

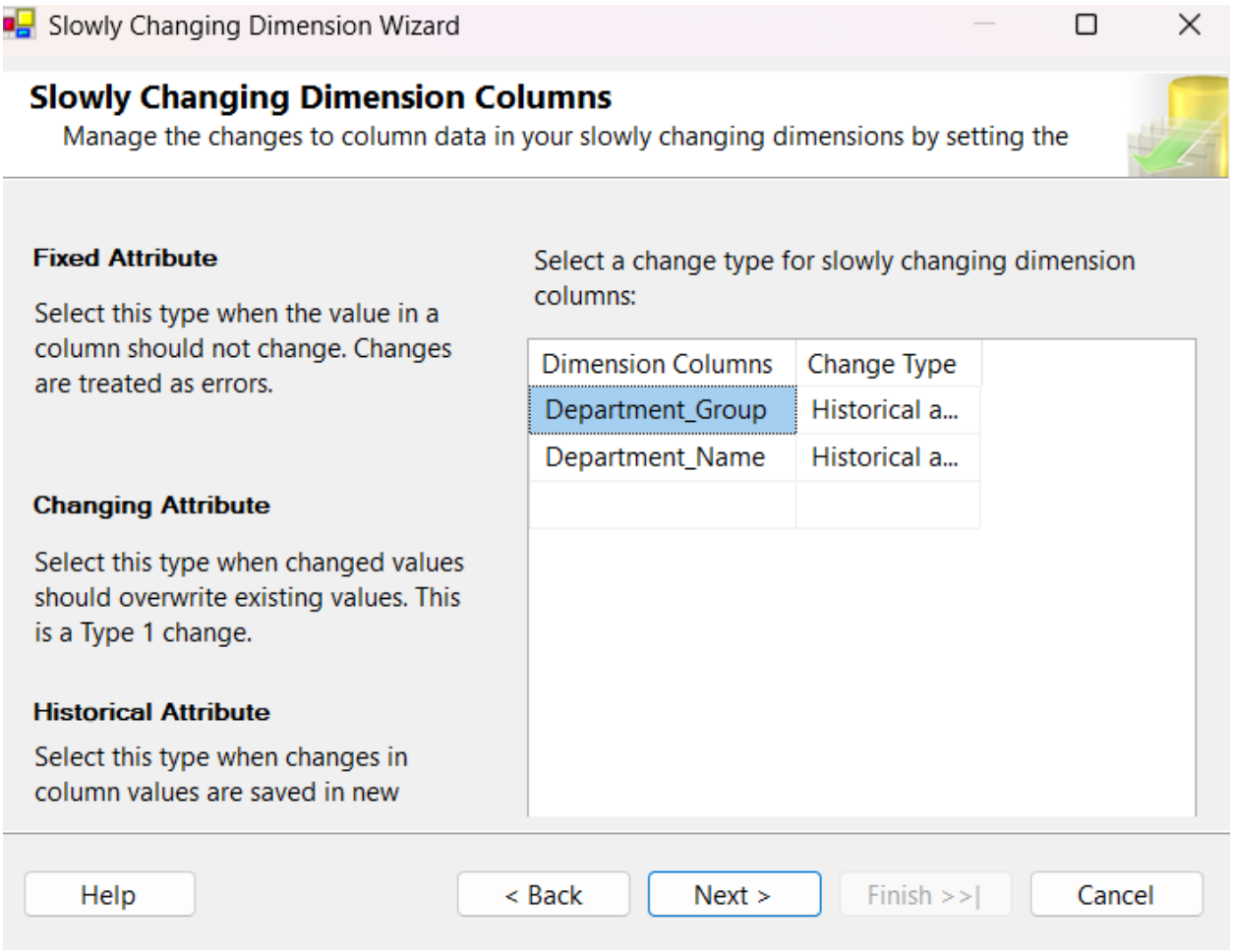


Figure 5.9: SCD Wizard - Business Key Selection

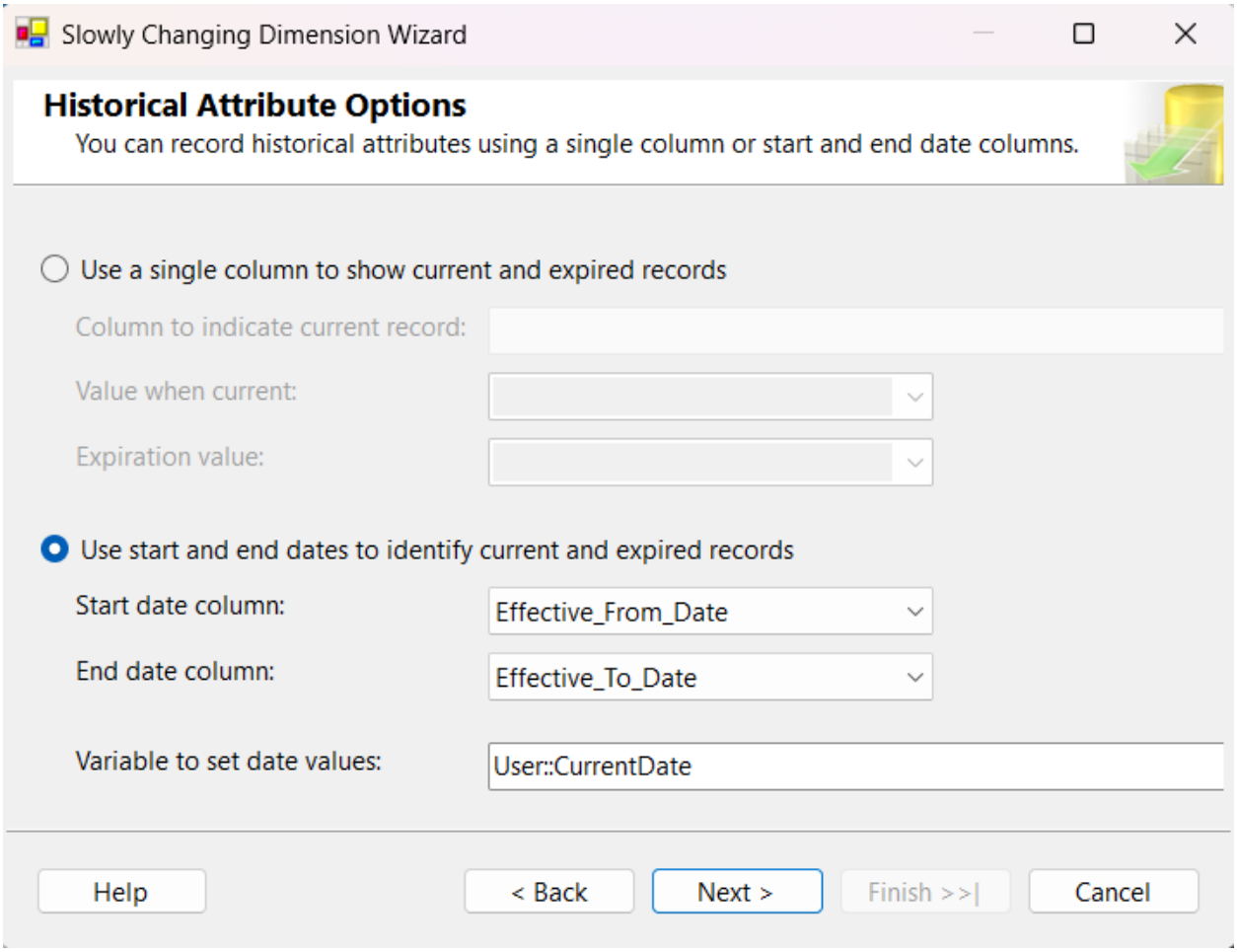


Figure 5.10: SCD Wizard - Historical Attribute Configuration

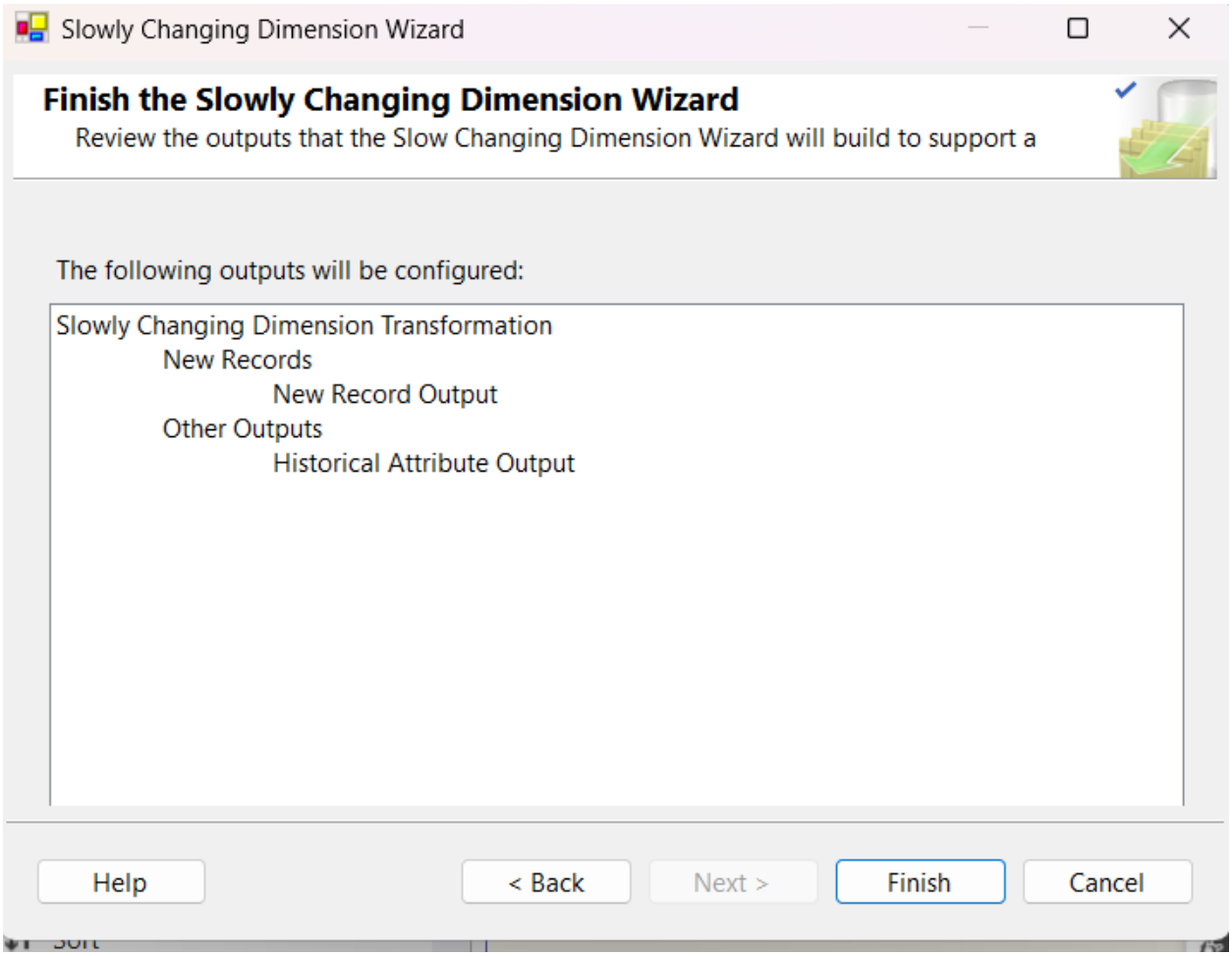


Figure 5.11: SCD Output Flow for New and Changed Records

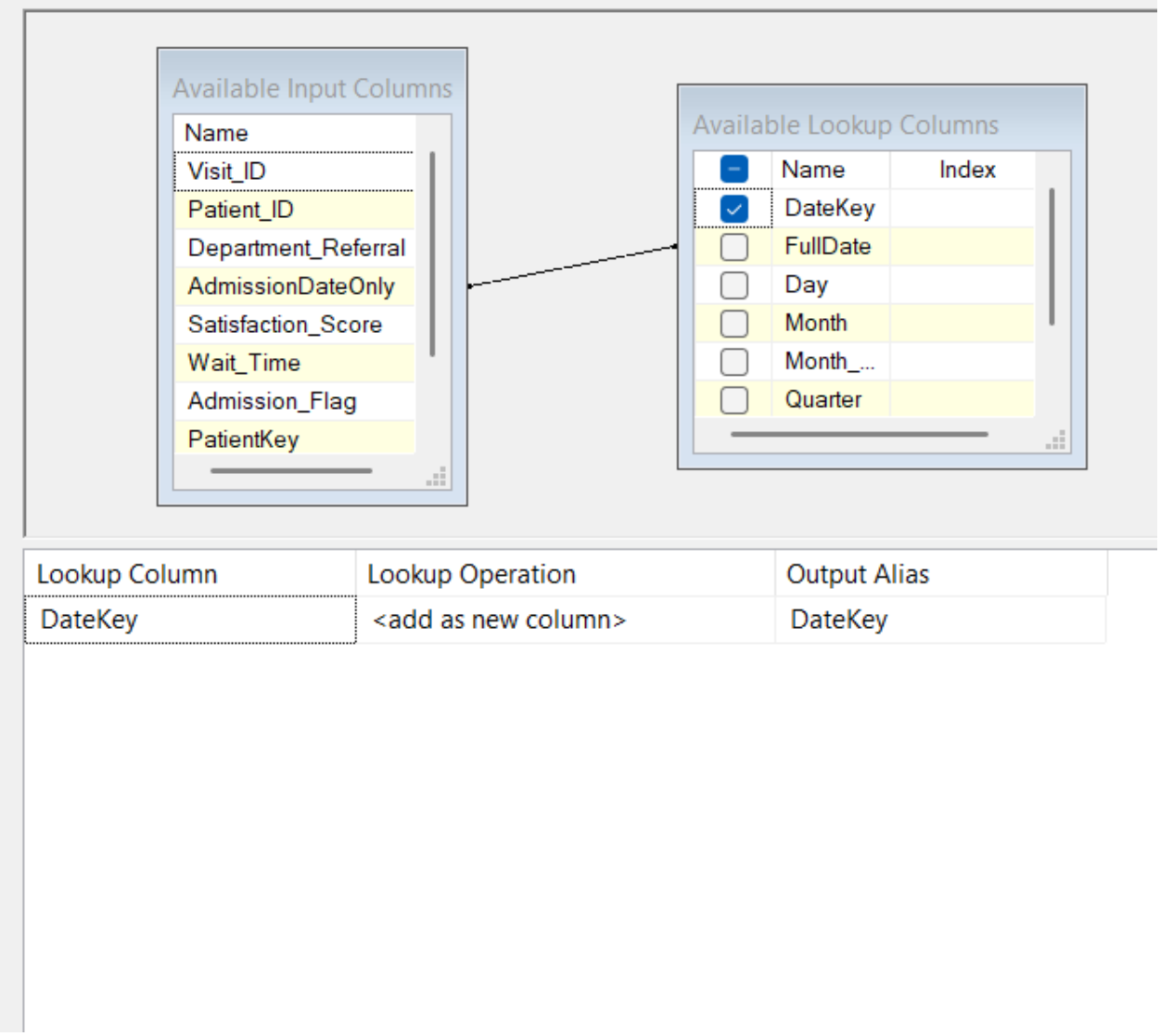


Figure 5.12: Successful Execution of Department SCD Load

5.8 Fact Table Loading

Fact table is loaded using Lookup transformations.

Steps:

- Lookup used to get surrogate keys:
  - Patient_Key
  - Department_Key
  - Date_Key
- Load into:
  - Fact_ER_Visit

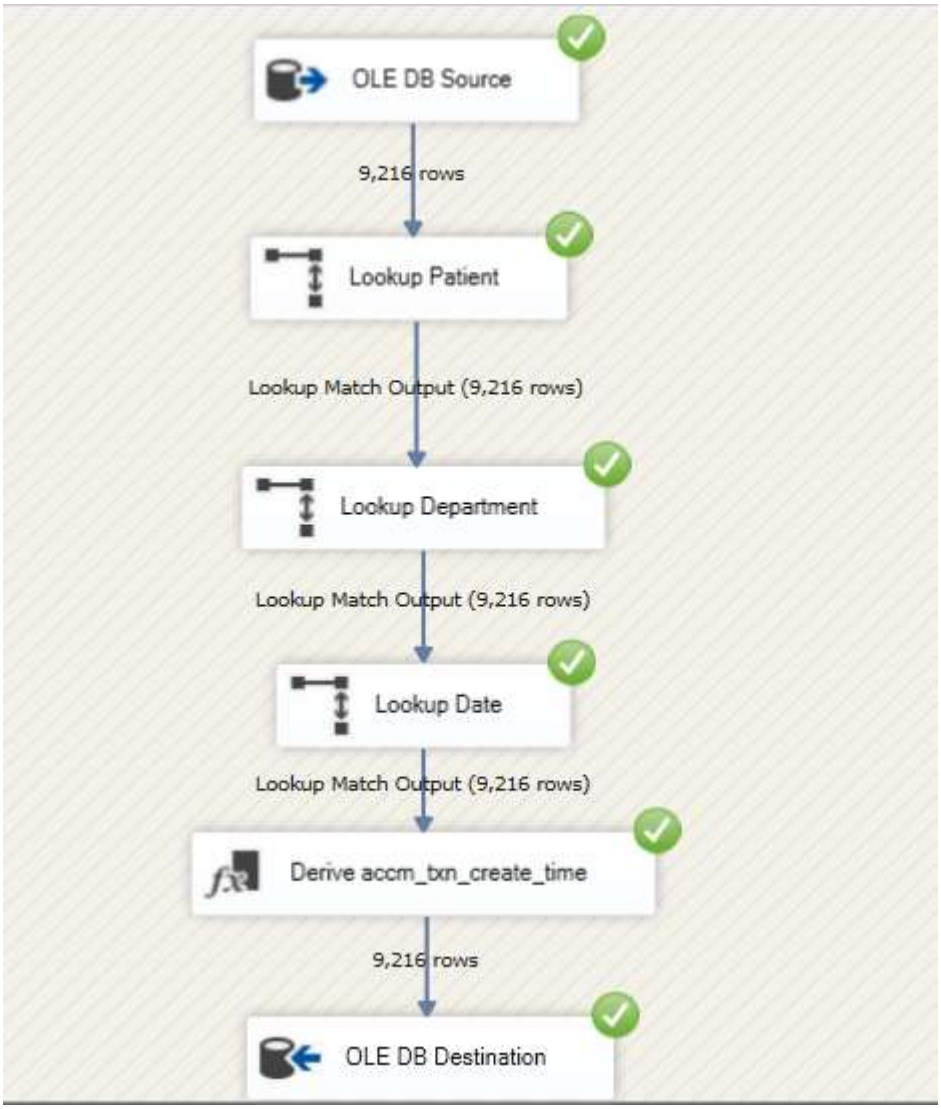
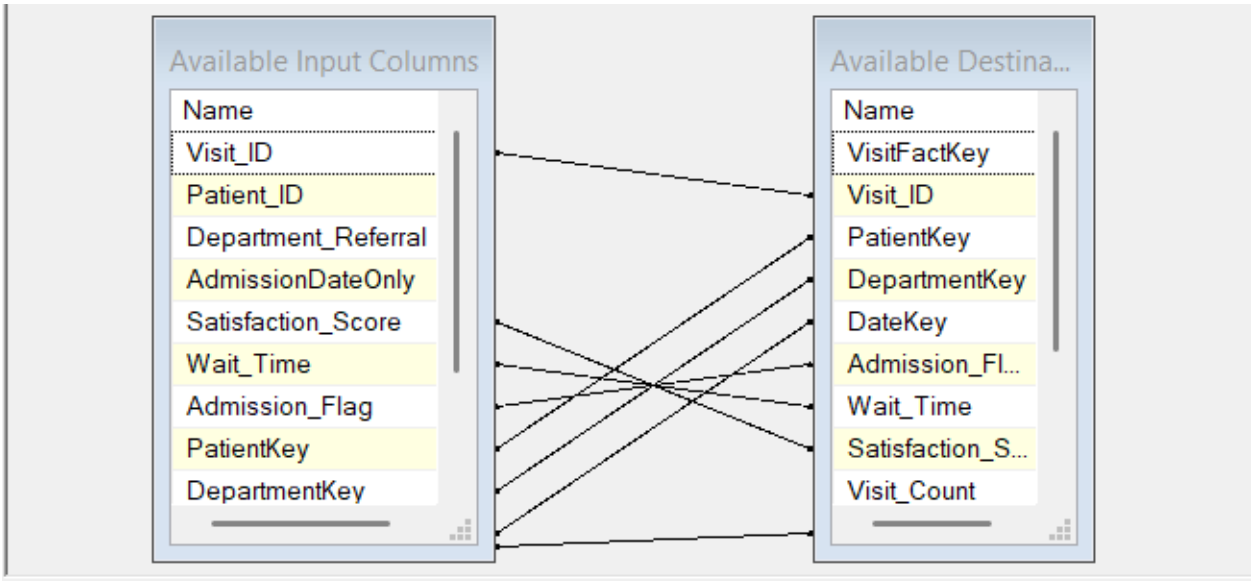


Figure 5.13: Lookup Transformation for Fact Table Loading

5.9 Control Flow Execution

The execution sequence ensures proper ETL flow:

1. Load staging tables
2. Load dimension tables
3. Apply SCD
4. Load fact table



Input Column	Destination Column
<ignore>	VisitFactKey
Visit_ID	Visit_ID
PatientKey	PatientKey
DepartmentKey	DepartmentKey
DateKey	DateKey
Admission_Flag	Admission_Flag
Wait_Time	Wait_Time
Satisfaction_Score	Satisfaction_Score
<ignore>	Visit_Count
accm_txn_create_time	accm_txn_create_time

Figure 5.14: Successful Execution of the Master ETL Package

5.10 Validation SQL Queries

```
SELECT COUNT(*) AS Patient_Staging_Count FROM Stg_Patient;
SELECT COUNT(*) AS ER_Visit_Staging_Count FROM Stg_ER_Visit;
SELECT COUNT(*) AS Department_Staging_Count FROM Stg_Department;
```

108 %

Results | Messages

	Patient_Staging_Count
1	73728

	ER_Visit_Staging_Count
1	55296

	Department_Staging_Count
1	48

Figure 5.15: SQL validation query showing staging table row counts

```
SELECT COUNT(*) AS Dim_Patient_Count FROM Dim_Patient;
SELECT COUNT(*) AS Dim_Department_Count FROM Dim_Department;
SELECT COUNT(*) AS Dim_Date_Count FROM Dim_Date;
```

108 %

Results | Messages

	Dim_Patient_Count
1	92160

	Dim_Department_Count
1	10

	Dim_Date_Count
1	579

Figure 5.16: SQL validation query showing dimension and fact table row counts

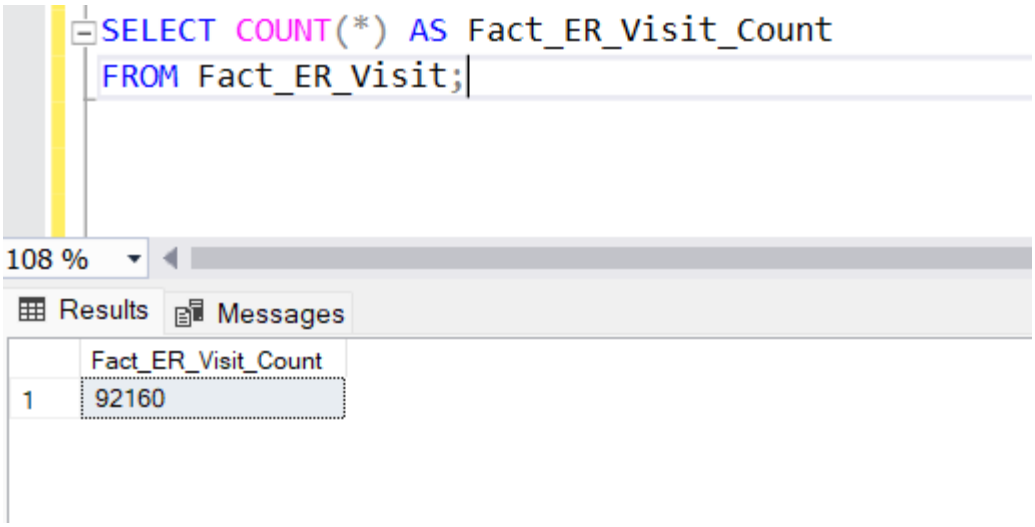


Figure 5.17: SQL validation query verifying surrogate key mapping between fact and dimension tables

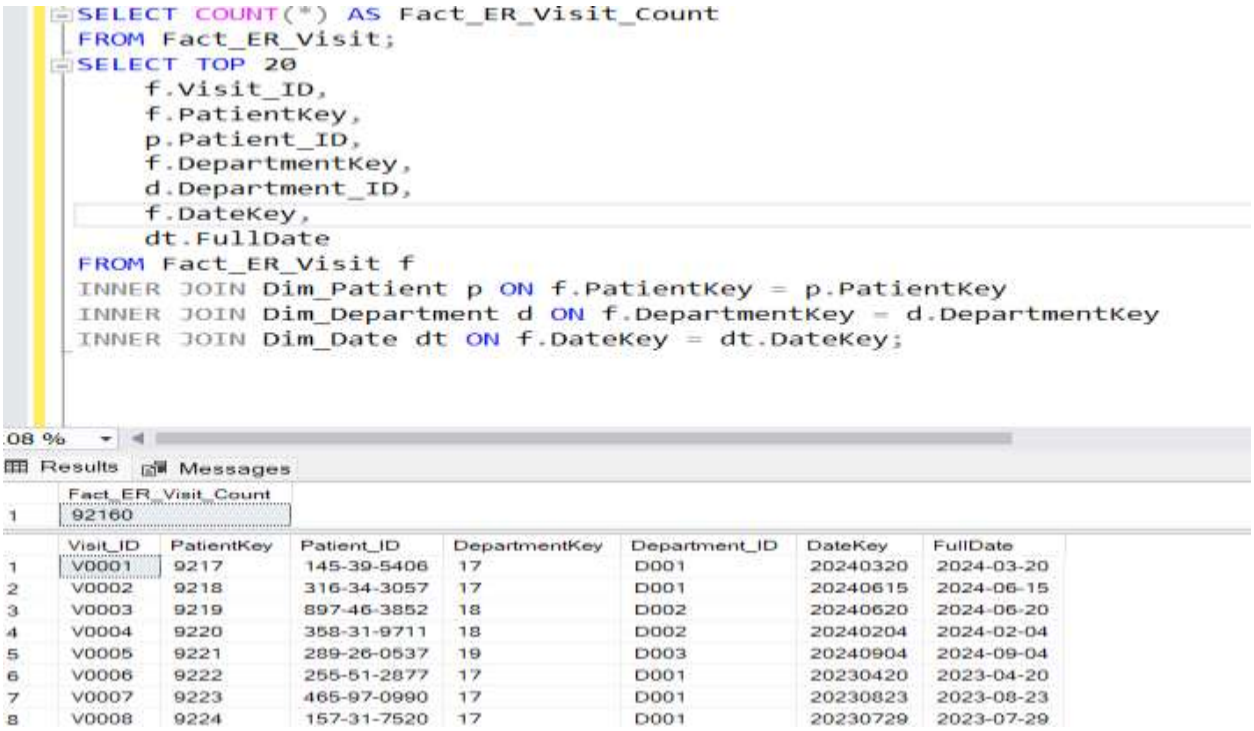


Figure 5.18: SQL validation query checking for missing foreign key references in Fact_ER_Visit

5.11 Error Handling in ETL Process

Error handling was incorporated into the ETL process to manage issues such as invalid data types, missing lookup matches, and null values. Data Conversion transformations were used to correct source data types, while Lookup transformations ensured that only valid dimension references were loaded into the fact table. The staging area also helped isolate source data and made debugging easier during ETL execution.

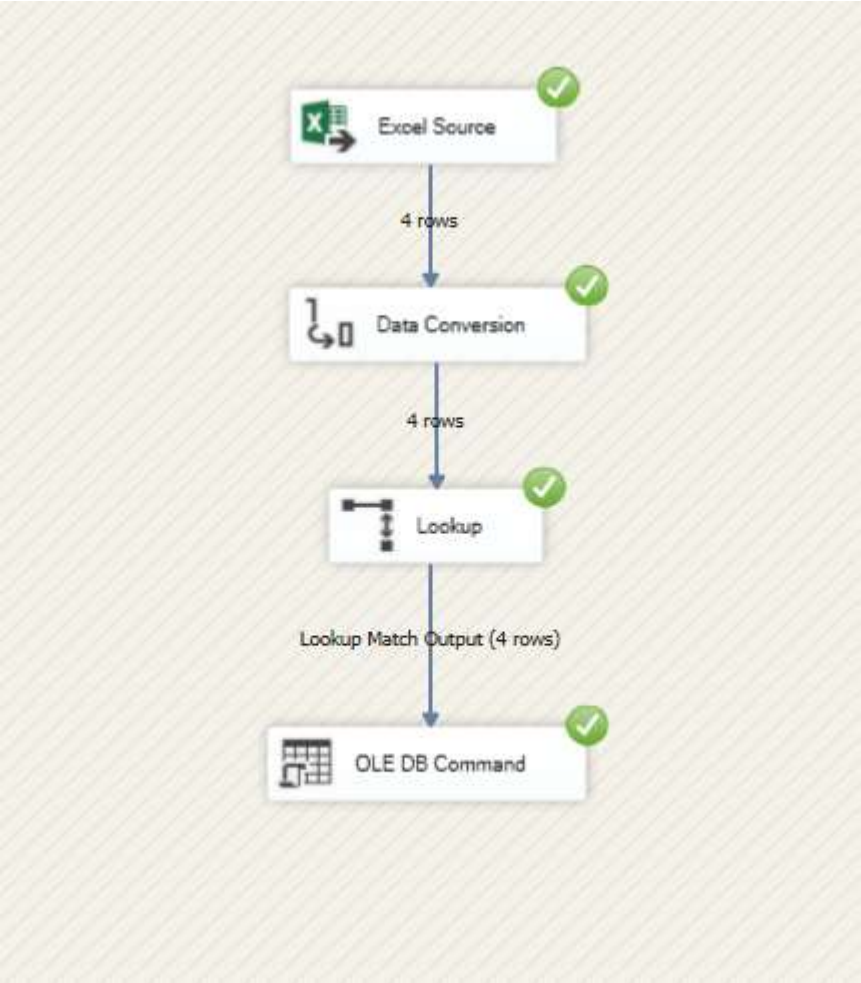


Figure 5.19: Lookup transformation configured to handle unmatched records using error output redirection

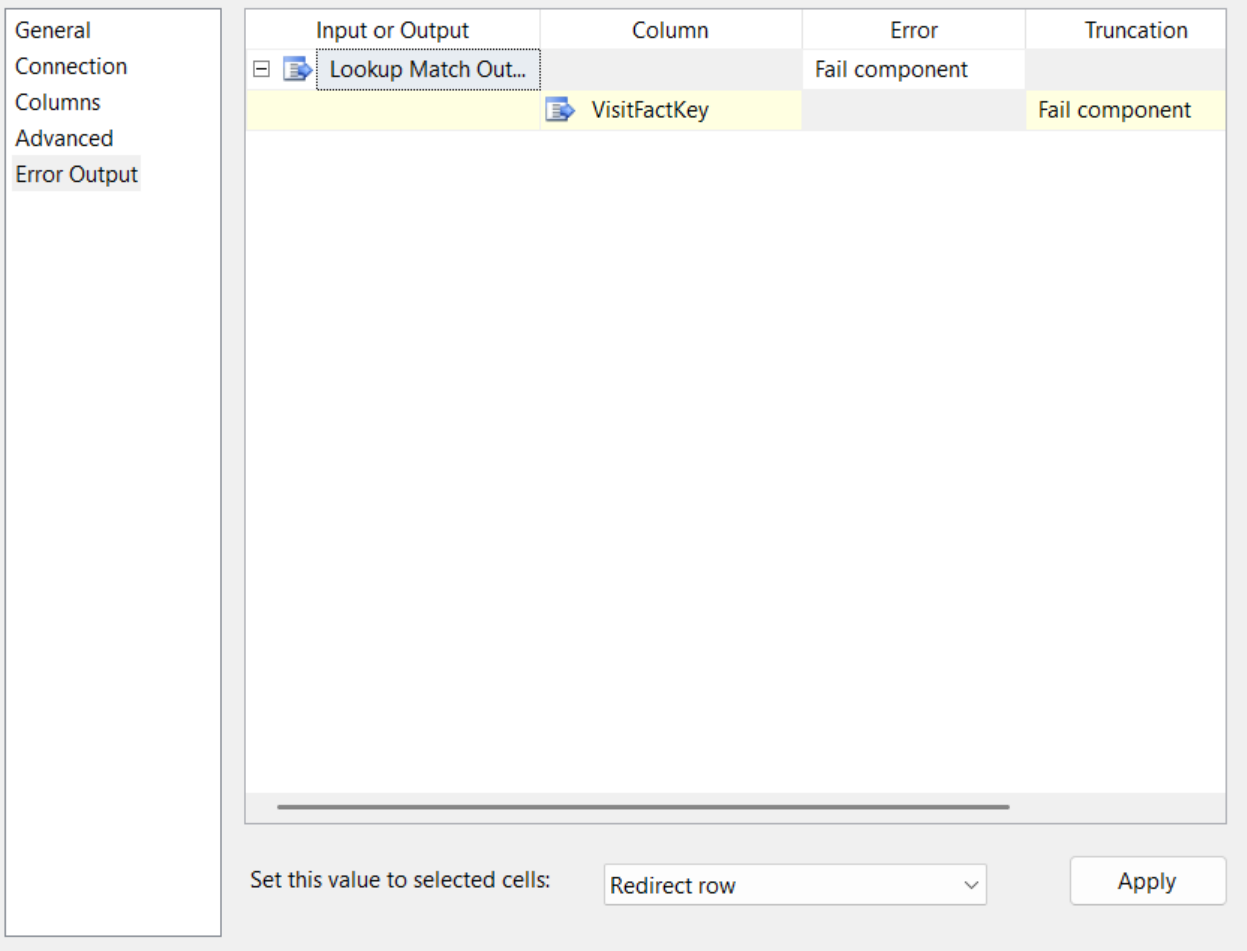


Figure 6.11: Data flow showing successful execution of accumulating fact table update process

5.11 Summary of ETL Process

The ETL process successfully:

- Integrated multiple data sources
- Cleaned and transformed data
- Loaded dimension and fact tables
- Implemented Slowly Changing Dimension
- Maintained data consistency and integrity

TASK 6-ETL Development for Accumulating Fact Table

6.1 Introduction

Task 6 focuses on extending the fact table to support an **accumulating fact table** design. Unlike a normal transactional fact table, an accumulating fact table stores the progress of a business event over time by updating the same fact row when new milestone data becomes available.

In this project, the business event is an **ER visit**. During the initial fact load, each ER visit record is inserted into `Fact_ER_Visit`. At that point, the row creation time is stored. Later, when completion information becomes available from a separate data source, the corresponding fact row is updated with the visit completion time and the total processing duration in hours.

This design satisfies the assignment requirement to implement a separate ETL pipeline for updating an accumulating fact table

6.2 Concept of Accumulating Fact Table

An accumulating fact table tracks the lifecycle of a transaction through milestone dates or timestamps. In this assignment, two important milestones are tracked for each ER visit:

- **Visit created** – when the fact row is first loaded into the warehouse
- **Visit completed** – when the separate completion dataset provides the final completion timestamp

A derived measure is then calculated:

- **Transaction processing time in hours** = `accm_txn_complete_time - accm_txn_create_time`

This approach is similar to the sample reports, where the fact row is first inserted with a create time and later updated through a dedicated SSIS package.

6.3 Extending the Fact Table

To support the accumulating fact logic, the `Fact_ER_Visit` table was extended with the following additional columns:

- `accm_txn_create_time` – stores the timestamp when the record is initially loaded
- `accm_txn_complete_time` – stores the timestamp when the ER visit is completed
- `txn_process_time_hours` – stores the difference in hours between the create and complete timestamps

```
-- ========================================
-- TASK 6 : ACCUMULATING FACT EXTENSIONS
-- ========================================
ALTER TABLE dbo.Fact_ER_Visit
ADD
    accm_txn_create_time DATETIME NULL,
    accm_txn_complete_time DATETIME NULL,
    txn_process_time_hours INT NULL;
```

Figure 6.1: SQL script used to extend Fact_ER_Visit with accumulating fact columns

6.4 Initial Loading Logic for Create Time

During the initial fact load in Task 5, the `accm_txn_create_time` column should be populated with the current system timestamp using a **Derived Column** transformation in SSIS. At this stage:

- `accm_txn_create_time = GETDATE()`
- `accm_txn_complete_time = NULL`
- `txn_process_time_hours = NULL`

Derived Column Expressions

Create these in the fact loading package:

```
UPDATE dbo.Fact_ER_Visit
SET txn_process_time_hours = DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time)
WHERE accm_txn_create_time IS NOT NULL
  AND accm_txn_complete_time IS NOT NULL;
```

Figure 6.2: Derived Column transformation used to initialize accumulating fact attributes during initial fact load

6.5 Preparation of Separate Completion Dataset

	A	B	C
1	Visit_ID	accm_txn_complete_time	
2	V0001	3/27/2026 10:00	
3	V0002	3/28/2026 11:30	
4	V0003	3/29/2026 12:15	
5	V0004	3/30/2026 13:00	
6			
7			

Figure 6.3: Completion update Excel file containing Visit_ID and accm_txn_complete_time

6.6 Separate SSIS Package for Task 6

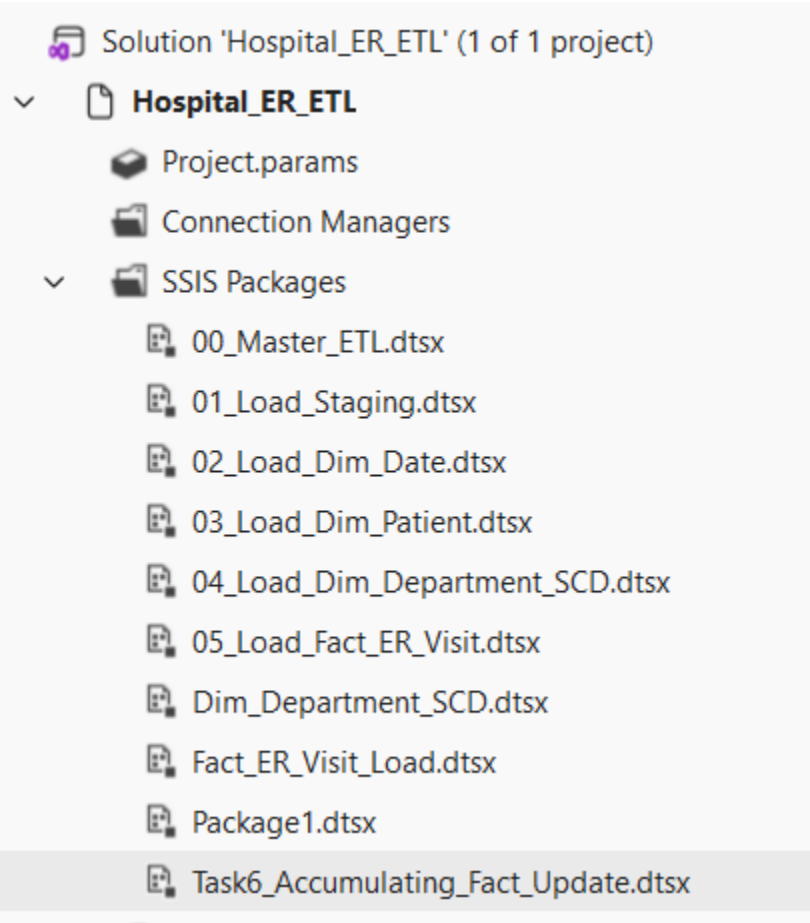


Figure 6.4: Separate SSIS package created for accumulating fact table update

6.7 Connection Managers Used

The following connection managers are required in this package:

- **HospitalDW_Conn** – OLE DB connection to the warehouse database
- **CompletionExcel_Conn** – Excel connection to `ER_Visit_Completion.xlsx`

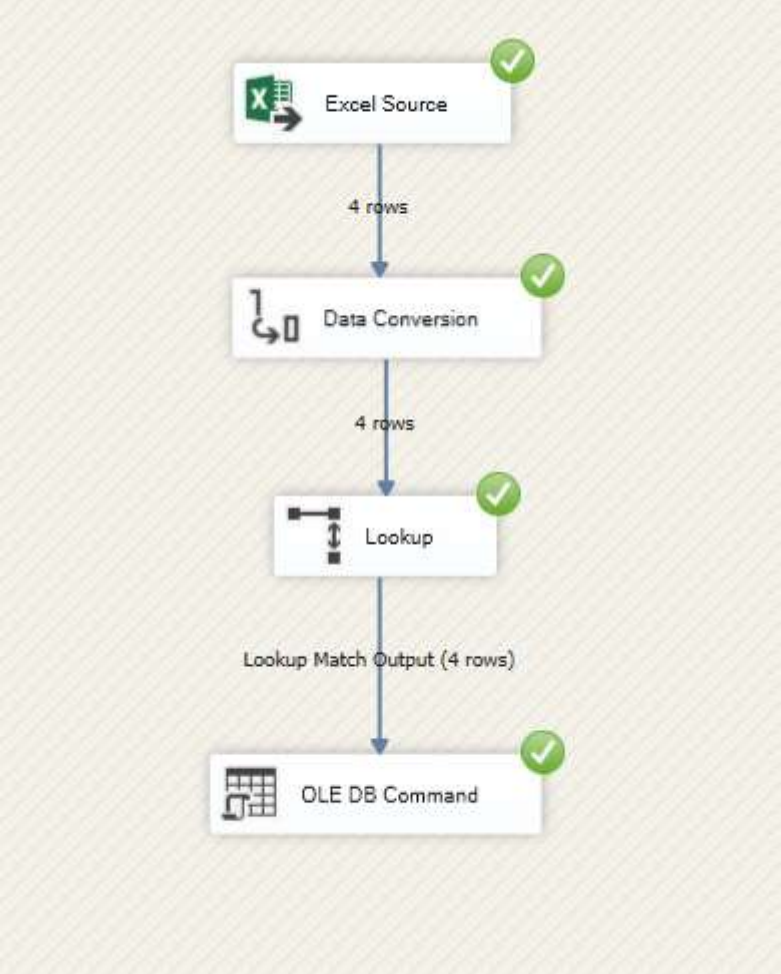


Figure 6.5: Connection managers configured for the accumulating fact update package

6.8 Control Flow design

The control flow for this package contains one main **Data Flow Task**:

- **Update Accumulating Fact Table**

You may also optionally add an **Execute SQL Task** before the Data Flow Task to check whether the fact table contains rows.

6.9.1 EXCEL Source

The Excel Source reads the completion dataset from `ER_Visit_Completion.xlsx`.

Configuration

- Data access mode: Table or view
- Select sheet containing:
  - `Visit_ID`
  - `accm_txn_complete_time`

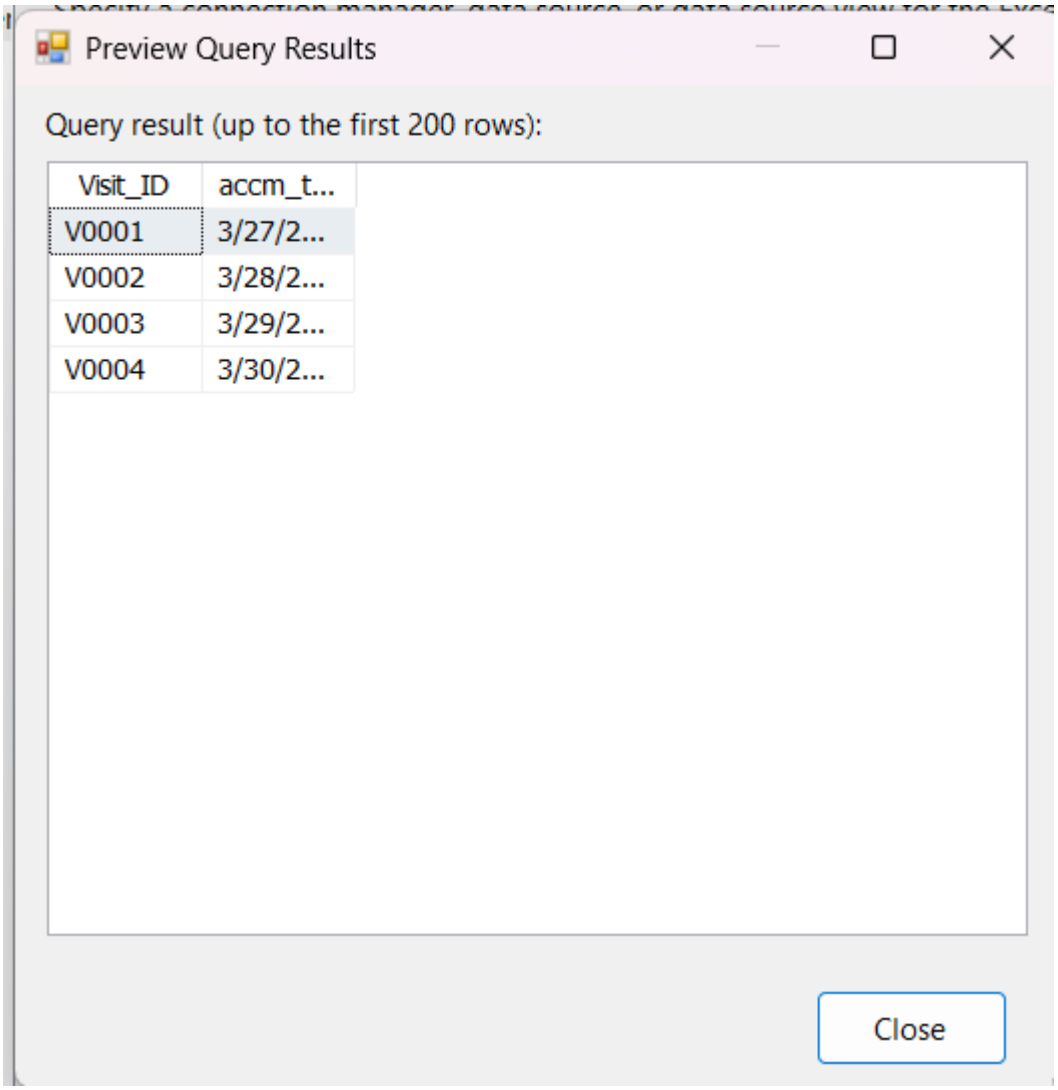


Figure 6.7: Excel Source configured to read ER visit completion update data

6.9.2 Data Conversion Transformation

The completion timestamp is read as text from Excel, so it must be converted to a database timestamp format before update.

Conversion

- `accm_txn_complete_time` → `completion_time_conv`
- Data type: `database timestamp [DT_DBTIMESTAMP]`

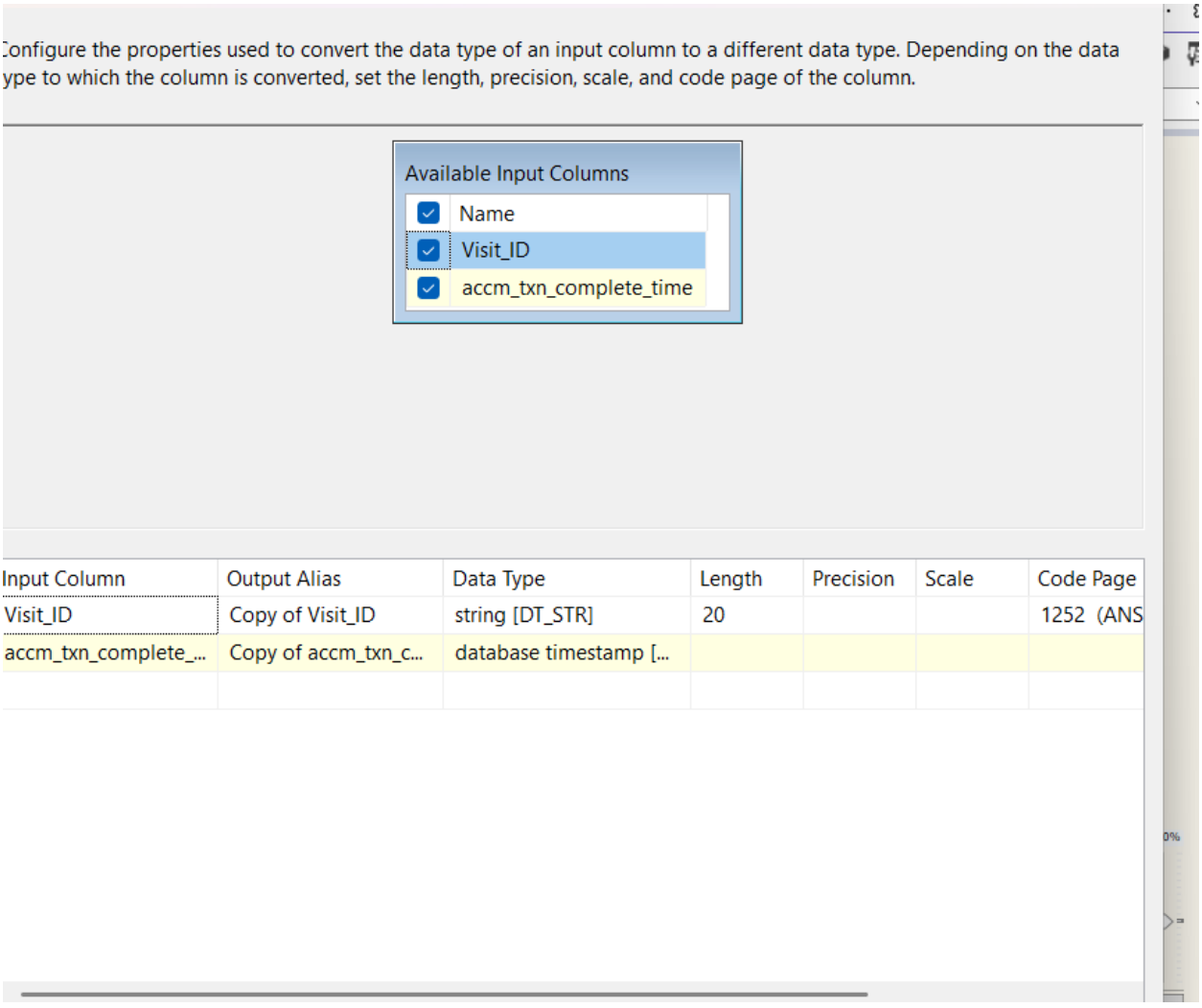


Figure 6.8: Data Conversion transformation converting completion time into database timestamp format

6.9.3 Lookup Transformaation

The Lookup transformation is used to match each Visit_ID from the completion file with the corresponding fact table record in Fact_ER_Visit.

Lookup Logic

- Input column: Visit_ID
- Lookup table: Fact_ER_Visit
- Match against: Visit_ID
- Return:
    - VisitFactKey or the fact table primary key if available
    - optionally accm_txn_create_time

You can match directly on Visit_ID, because it is the natural transaction key in your fact table.

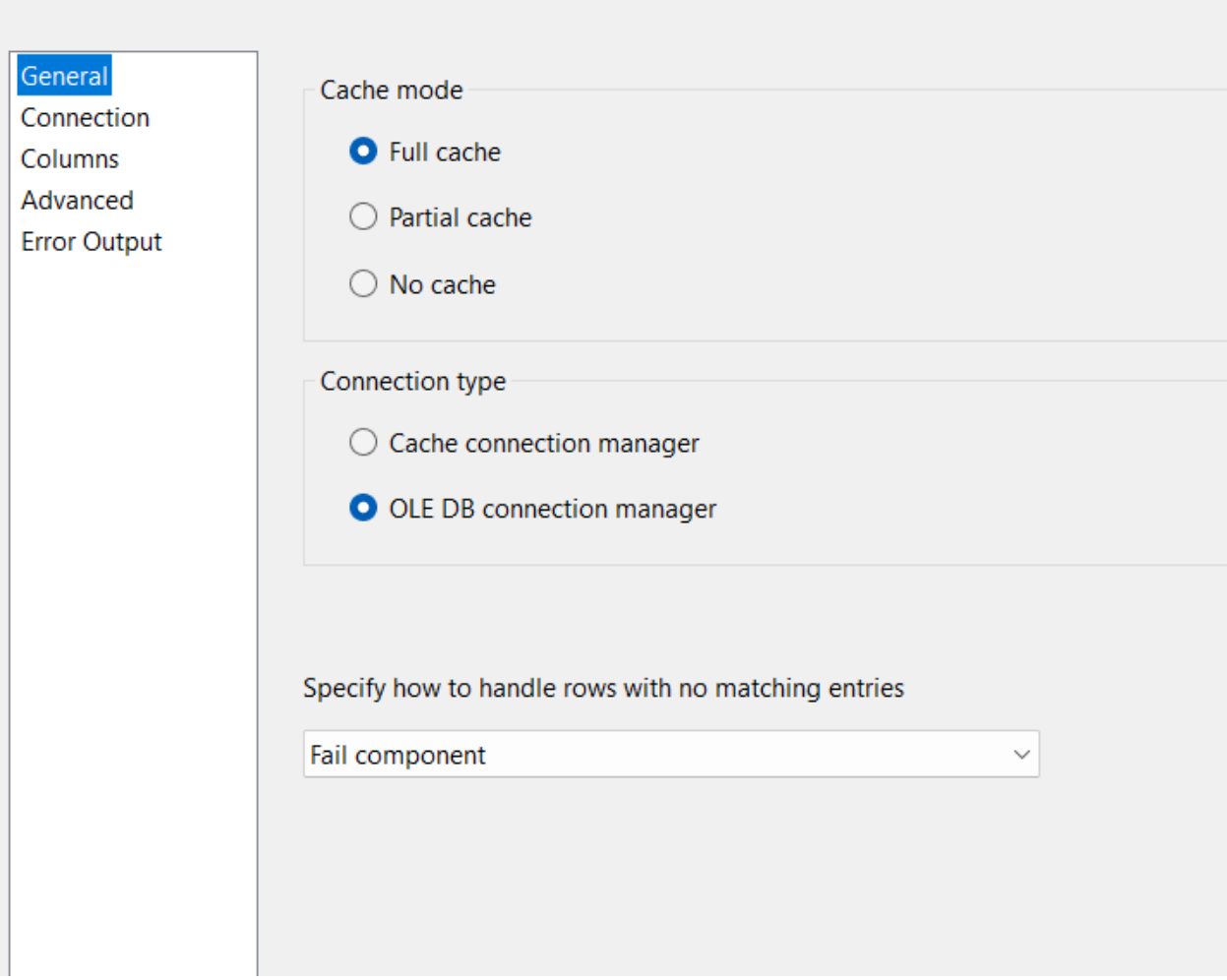


Figure 6.9.1 Lookup transformation matching completion dataset Visit_ID values with Fact_ER_Visit records

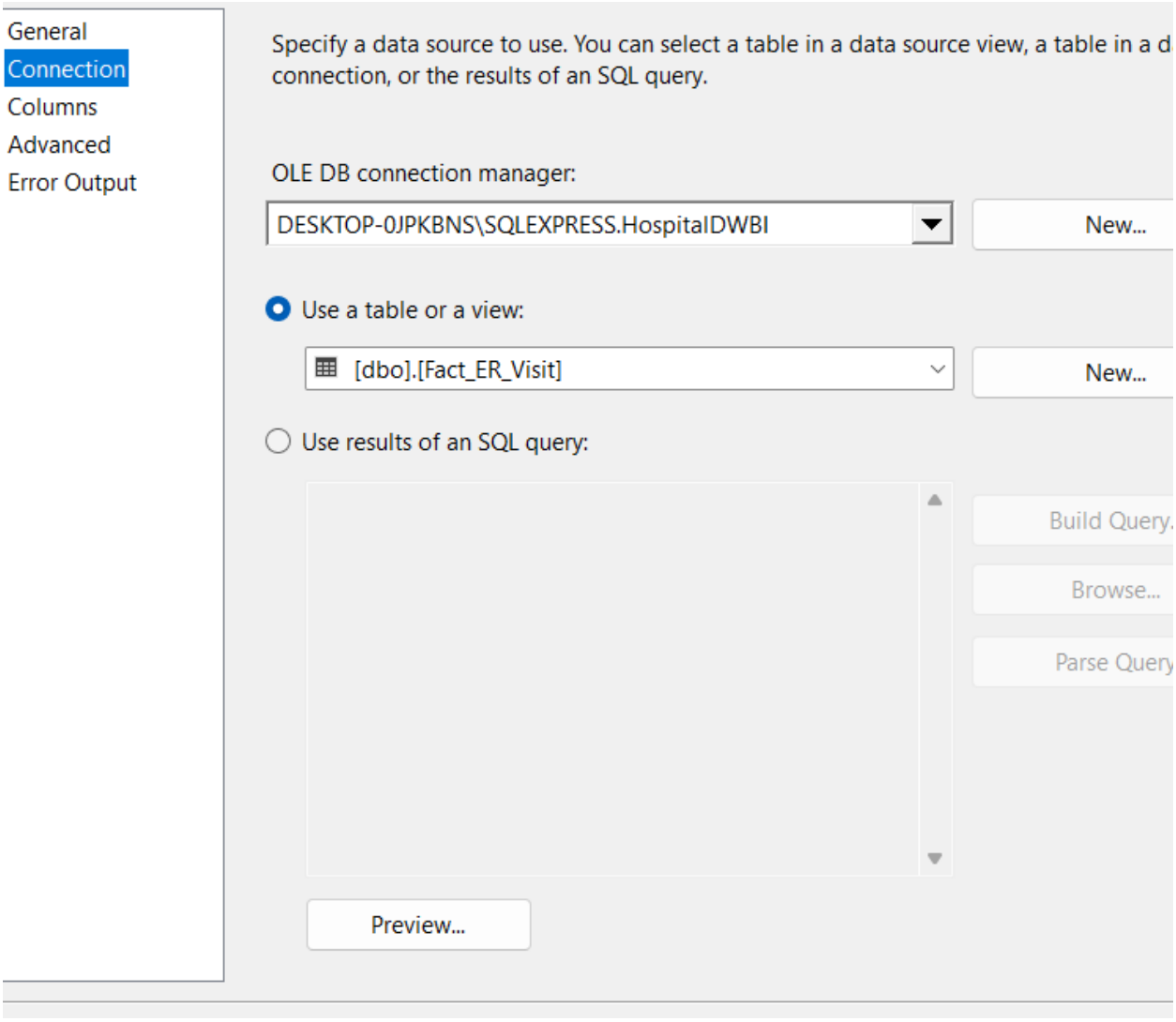


Figure 6.9.2: Lookup transformation matching completion dataset Visit_ID values with Fact_ER_Visit records

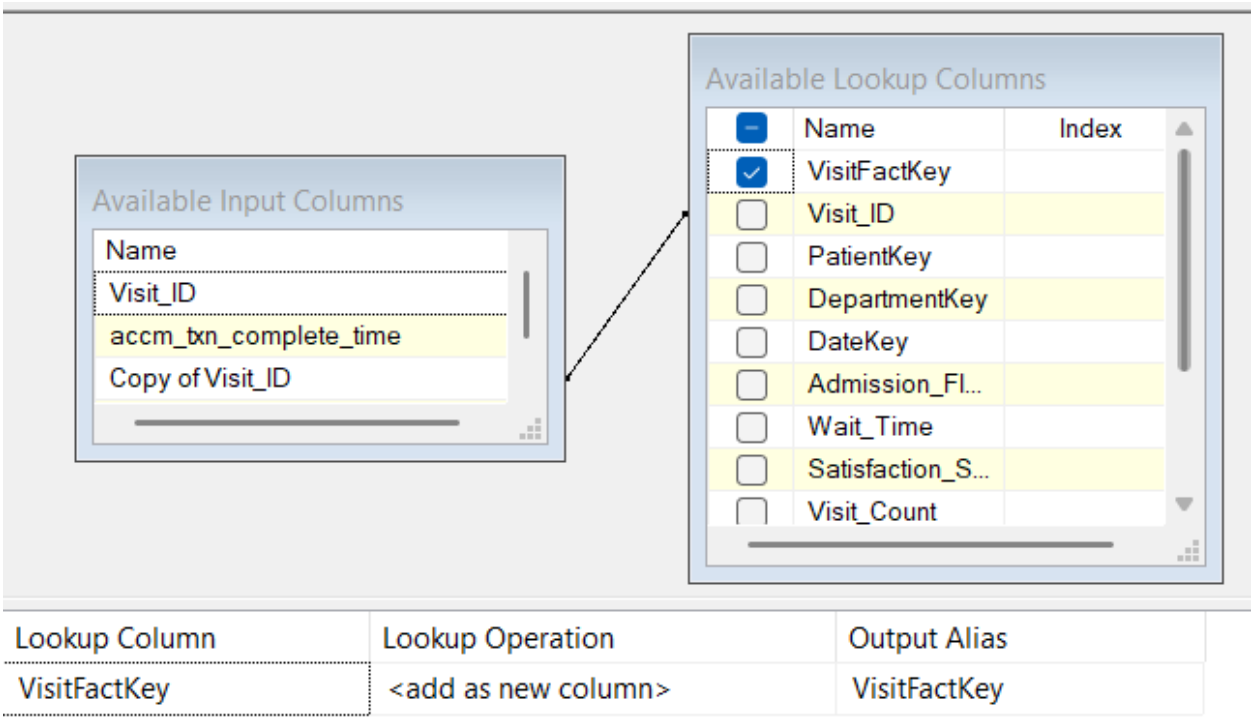


Figure 6.9:.3 Lookup transformation matching completion dataset Visit_ID values with Fact_ER_Visit records

6.9.4 OLE DB Command Transformation

The matched rows are sent to an **OLE DB Command** transformation, which performs a row-by-row update on the fact table.

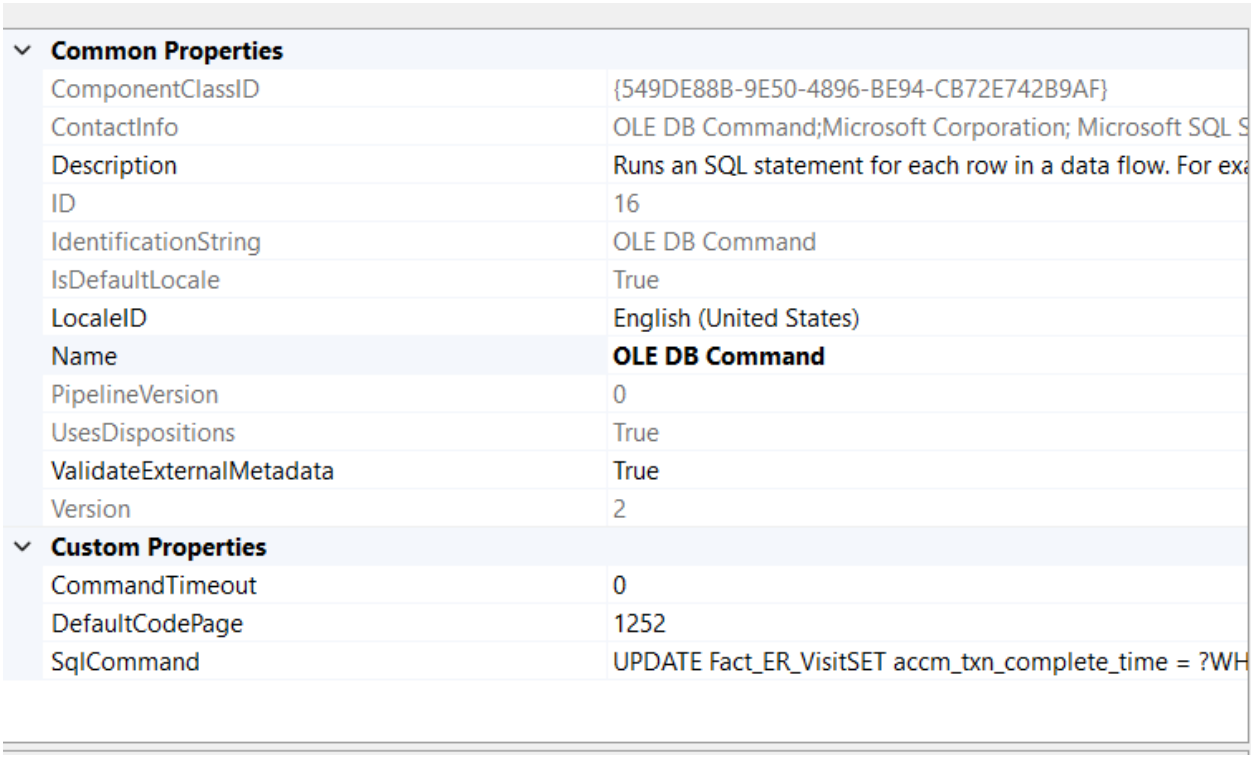


Figure 6.10.1: OLE DB Command used to update accm_txn_complete_time and txn_process_time_hours in Fact_ER_Visit

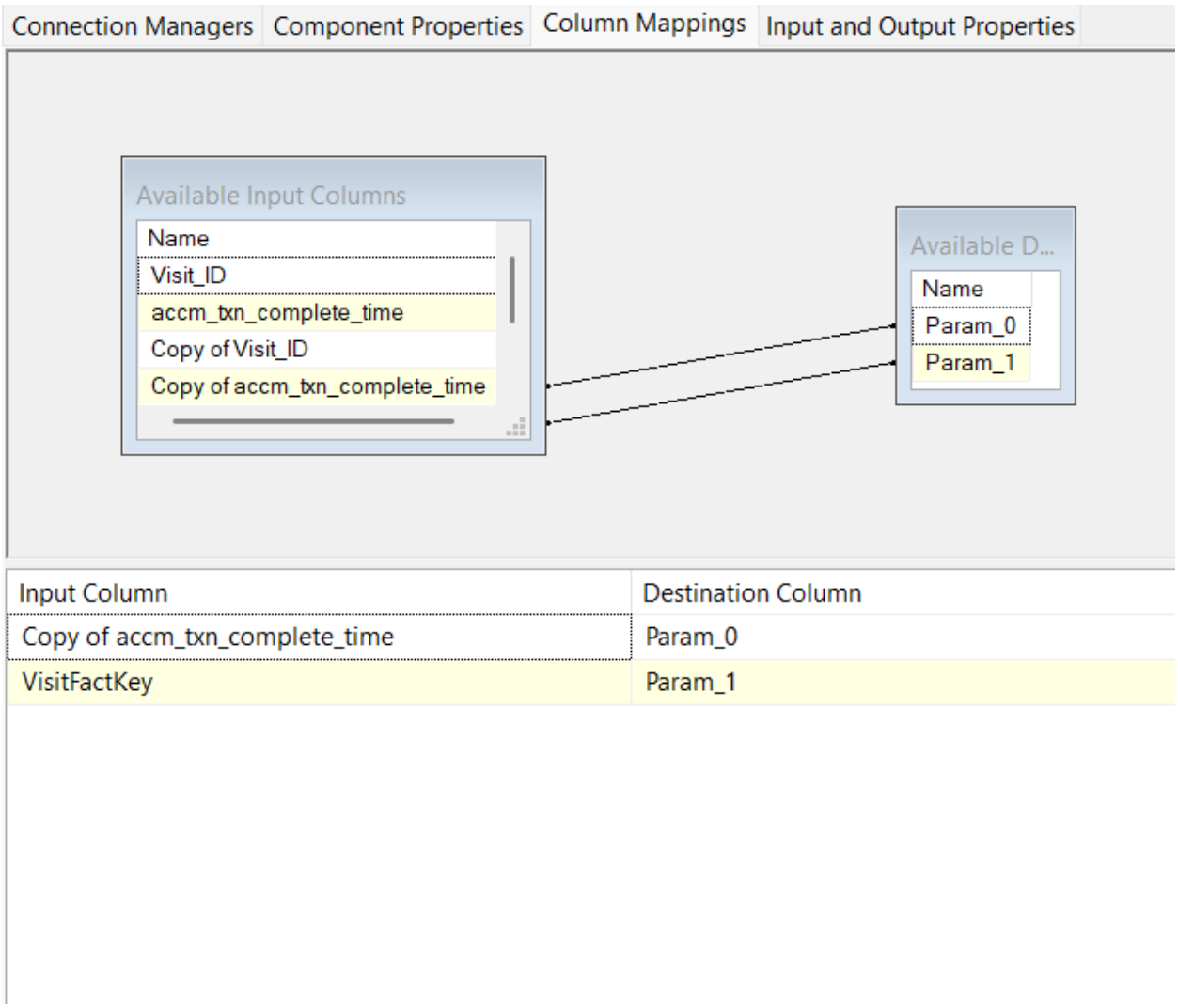


Figure 6.10.2: OLE DB Command used to update accm_txn_complete_time and txn_process_time_hours in Fact_ER_Visit

6.10 Execution of the Package

After configuring the package, it was executed successfully. Matching ER visits were updated in the fact table with:

- completion timestamp
- processing time in hours

Rows with non-matching `Visit_ID` values were redirected and did not stop the package execution.

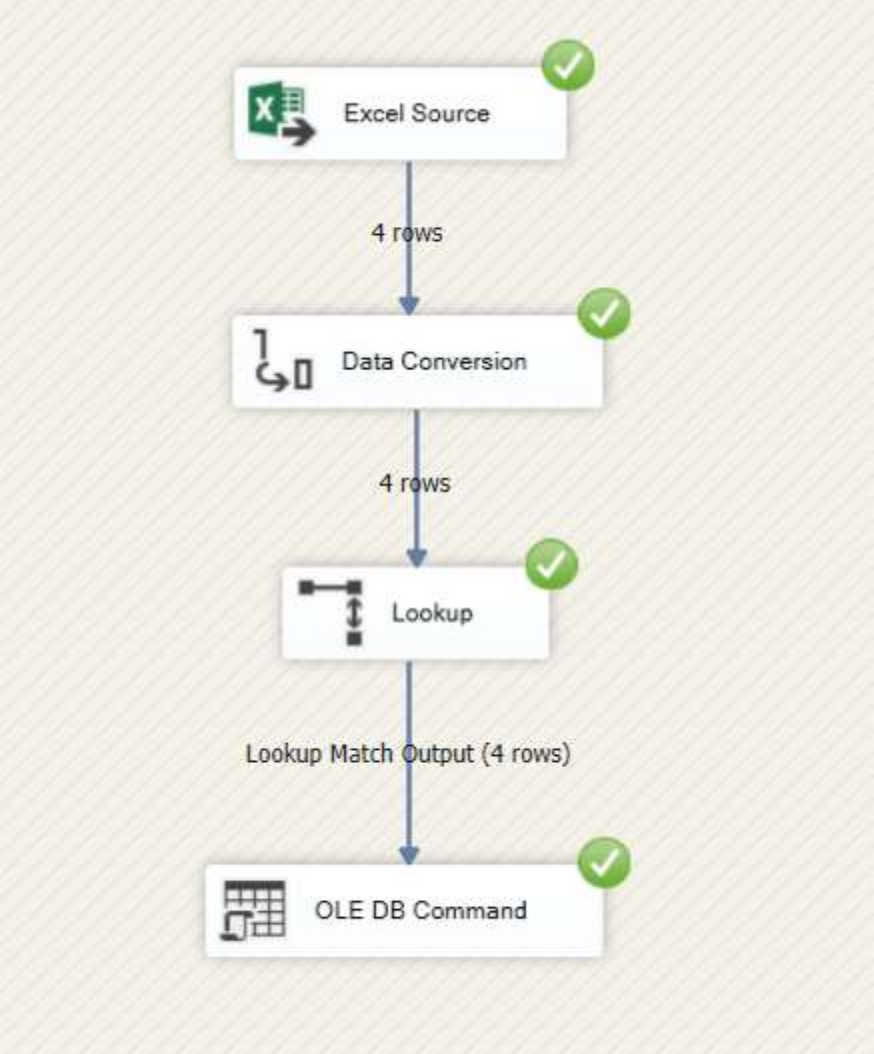


Figure 6.12: Successful execution of the accumulating fact update package

6.11 Validation Results

```
SELECT TOP 10
    Visit_ID,
    accm_txn_create_time,
    accm_txn_complete_time,
    txn_process_time_hours
FROM Fact_ER_Visit;

SELECT
    Visit_ID,
    accm_txn_create_time,
```

108 %

Results | Messages

	Visit_ID	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	V0001	2026-03-26 20:43:50.243	2026-03-27 10:00:00.000	14
2	V0002	2026-03-26 20:43:50.243	2026-03-28 11:30:00.000	39
3	V0003	2026-03-26 20:43:50.243	2026-03-29 12:15:00.000	64
4	V0004	2026-03-26 20:43:50.243	2026-03-30 13:00:00.000	89
5	V0005	2026-03-26 20:43:50.243	NULL	NULL
6	V0006	2026-03-26 20:43:50.243	NULL	NULL
7	V0007	2026-03-26 20:43:50.243	NULL	NULL
8	V0008	2026-03-26 20:43:50.243	NULL	NULL
9	V0009	2026-03-26 20:43:50.243	NULL	NULL
10	V0010	2026-03-26 20:43:50.243	NULL	NULL

Figure 6.13: Validation query showing ER visits updated with completion timestamps and calculated process hours

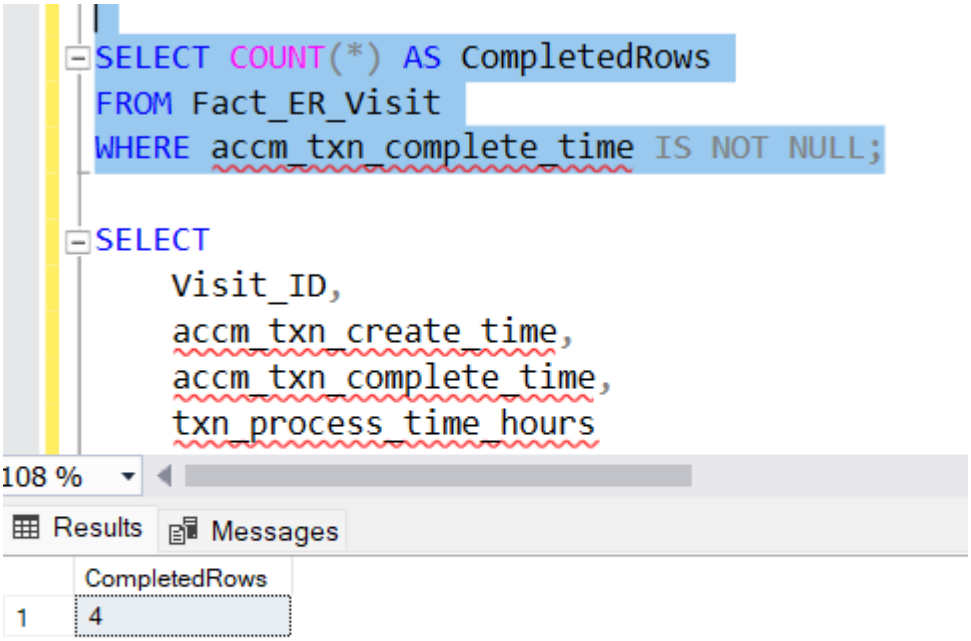


Figure 6.14: Validation query showing number of completed ER visits updated by the package

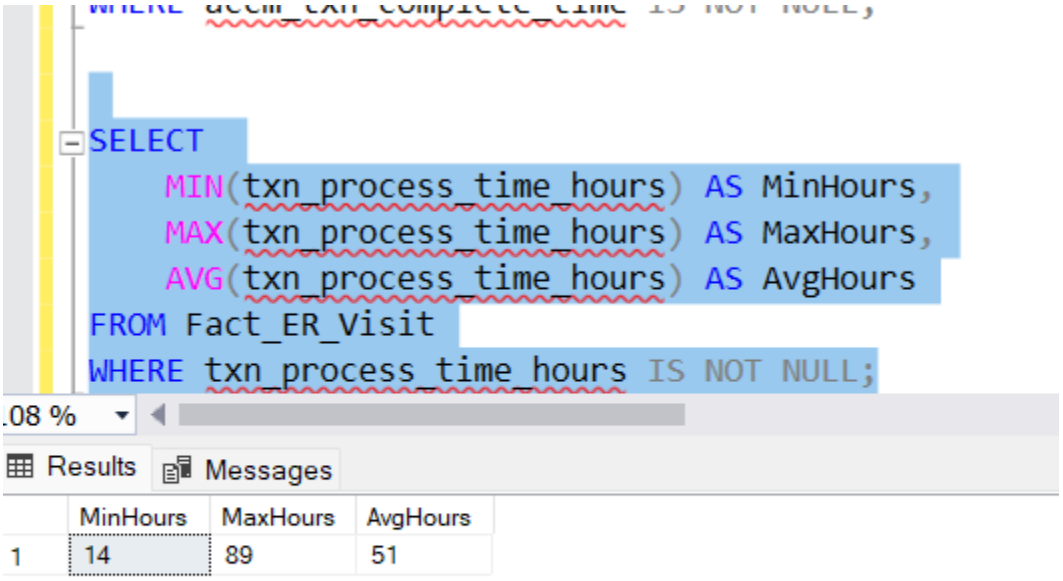


Figure 6.15: Validation query showing minimum, maximum, and average processing time hours

6.12 Conclusion

In Task 6, the Fact_ER_Visit table was successfully extended to support accumulating fact table behavior. The accm_txn_create_time was populated during the initial fact load, while a separate Excel completion dataset was later processed through a dedicated SSIS package to update accm_txn_complete_time and calculate txn_process_time_hours.

The use of a separate update package, Lookup transformation, OLE DB Command, and SQL validation queries demonstrates a complete and correct implementation of the accumulating fact table requirement. This fulfills the assignment requirement for Task 6 and strengthens the overall ETL solution.